

北京邮电大学

实验报告



题目： Linux 进程线程管理与同步互斥

班 级： 2023211303

学 号： 2023212872

姓 名： 计子毅

学 院： 计算机学院（国家示范性软件学院）

⋮

2025 年 12 月 18 日

Linux进程线程管理与同步互斥

1. 实验目的

2. 实验环境

3. 实验内容

- 3.1 进程创建与管理
- 3.2 线程创建管理与通信
- 3.3 进程通信
- 3.4 线程同步与互斥
- 3.5 多核多线程性能分析

4. 实验设计原理

- 4.1 进程管理模块设计原理
- 4.2 线程管理模块设计原理
- 4.3 进程通信模块设计原理
- 4.4 线程通信模块设计原理
- 4.5 线程同步与互斥模块设计原理
- 4.6 多核多线程性能分析设计原理

5. 实验步骤

- 5.1 环境准备
- 5.2 第一组 进程的创建和管理
- 5.3 第二组 线程的创建和管理
- 5.4 第三组 进程间通信
- 5.5 第四组 线程间通信
- 5.6 第五组 同步与互斥
- 5.7 第六组 多核多线程编程及性能分析
- 5.8 运行及测试验证
- 5.9 性能分析

6. 实验结果及分析

- 6.1 进程管理实验结果
- 6.2 线程管理实验结果
- 6.3 进程通信实验结果
- 6.4 线程通信实验结果
- 6.5 同步互斥实验结果
- 6.6 性能分析结果

7. 实验总结

- 7.1 主要收获
- 7.2 遇到问题

附录：主要程序代码

- 实验1.2 进程创建与管理 lab1.c
- 实验1.3 进程管理命令 lab1-3.sh
- 实验2.2 线程创建与管理 lab2.c
- 实验3 进程通信 lab3.c
- 实验4 线程通信 lab4.c
- 实验5 同步与互斥 lab5.c
- 实验6-1 观察CPU lab6-1.c
- 实验6-2 单线程/进程串行 vs 2 线程并行 vs 3 线程加锁并行程序对比 lab6-2.c
- 实验6-3 线程加锁 vs 3 线程不加锁 lab6-3.c
- 实验6-4 针对 Cache 的优化 lab6-4.c
- 实验6-5 CPU 亲和力对并行程序影响 lab6-5.c
- 启动脚本 run.sh

Linux进程线程管理与同步互斥

1. 实验目的

1. 以 Linux 操作系统为实验对象，理解 Linux 系统中任务（task）和进程、线程等概念，掌握利用 Linux API 函数/系统调用创建和管理进程的方法，认识进程生命周期、状态转换和并发执行的实质。
2. 理解进程和内核级、用户级线程的概念，掌握在 Linux 操作系统环境下，采用 Pthread线程库创建和管理线程的方法，观察分析线程并发执行和通信行为。
3. 掌握 Linux 提供的消息队列、共享内存、管道、signal 信号（软中断）四种进程间通信的原理和方法，采用系统调用和库函数编程实现这三种进程间通信机制。
4. 掌握 Linux 提供的内核信号量、用户态信号量的原理和方法，以及系统调用和 API 函数，采用这两类进程/线程同步互斥方式机制，针对生产者-消费者、哲学家就餐、睡眠理发师等经典同步互斥问题的扩展问题，设计并编程实现解决方案，观察验证方案正确性，加深对复杂进程线程同步互斥问题的理解。或：了解C++11等程序设计语言提供的线程同步互斥机制（类似于教科书中管程monitor），掌握使用互斥变量和条件变量实现多线程间的同步互斥的方法，编程实现上述典型同步互斥问题。
5. 通过 Linux 环境下多核多线程编程，定量分析线程自身业务逻辑、并发线程数目、线程间同步互斥、CPU 亲和、cache 优化对多线程并发/并行线程执行效率和系统吞吐量的影响，初步掌握针对复杂任务的多核多线程编程及性能分析方法。

2. 实验环境

- **硬件环境：**
物理CPU数量: 1
每个CPU的核数: 10
逻辑CPU数量: 16
CPU型号: model name : 12th Gen Intel(R) Core(TM) i7-12650H
- **软件环境：**Linux操作系统（Ubuntu发行版24.04），gcc编译器，Pthread线程库
- **开发语言：**C

3. 实验内容

3.1 进程创建与管理

- 参照 Linux 内核源码结构，阅读 Linux 内核源码，分析 Linux 进程的组成，观察 Linux 进程的 task_struct 等进程管理数据结构；
- 参照相关示例程序，查阅 Linux 系统调用等相关资料，设计父进程和子进程的业务处理逻辑；利用 C++等高级程序设计语言和 fork()、clone、exec()、wait()、exit()、kill()、getpid()等系统调用，创建管理进程，观察父子进程/线程的结构和并发行为，掌握等待、撤销等进程控制方法。

要求：

- 至少用到 fork()、clone、exec、wait、exit、kill、getpid 等 6 个系统调用；
- 所创建的父子进程各自具有不同的业务处理逻辑，父进程创建子进程后，子进程通过exec()调入自身执行代码；
- 进程可以自己通过 exit()主动结束，也可以被父进程执行 kill()命令来结束；

- 观察对比通过 fork()和 clone()创建的子任务/进程/线程的差异，分析 clone()系统调用中设置与不设置 CLONE_FS、CLONE_VFORK、CLONE_FILES、CLONE_FS、CLONE_PID等参数对所创建的子进程/线程的影响；
- 在创建的父亲进程/线程代码中的不同位置处增加随机延迟，使得进程执行横跨多个时间片，如通过增加数十到数百毫秒的延迟，保证进程执行时间不少于 3 个时间片。
- 掌握 ps、top、pstree -h、vmstat、strace、ltrace、sleep x、kill、jobs 等命令的功能和使用方式，利用这些命令观察进程的行为和特征。

3.2 线程创建管理与通信

• 线程创建与管理

查阅 Linux 和 Pthread 线程库相关资料，参照相关示例程序，设计父进程/线程和子线程的业务处理逻辑；利用 pthread_create、pthread_exit、pthread_cancel、pthread_join、pthread_self等线程管理函数，创建和管理线程，观察父子进程/线程的结构和并发行为，掌握等待、退出、撤销等线程控制方法。

要求：

- 在一个进程内创建多个子线程（如不少于 3 个子线程），父子子线程具有各自不同的业务处理逻辑，子线程执行自身特定的线程函数。父子线程间通过 pthread_join()函数实现同步和资源释放；
- 一个进程内的多个子线程分别以不同方式终止或退出执行，如通过 pthread_exit()和return 自己主动终止，或被其它线程通过 pthread_cancel 被动终止，观察对比以不同方式退出执行的子线程的行为差异；
- 在创建的父亲进程/线程代码中的不同位置处增加数十到数百毫秒的随机延迟（使用sleep()），使得进程和线程的执行横跨多个时间片，保证线程执行时间不少于 3 个时间片。

• 线程通信

属于同一个进程中的多个线程使用相同的地址空间，共享大部分数据，一个线程的数据可以直接为其它线程所用，这些线程相互间可以方便快捷地利用共享数据结构进行通信。编写程序，创建线程，实现主线程与子线程间、子线程相互间通过共享数据类型，如整型变量、字符串、结构体等传递信息，进行通信。

3.3 进程通信

该部分实验有四种通信方式可供实现，此处选择管道通信。

管道通信

管道是进程间共享的、用于相互间通信的文件，Linux/Unix 提供了2 种类型管道：

（1）用于父进程-子进程间通信的无名管道。无名管道是利用系统调用 pipe(filedes)创建的无路径名的临时文件，该系统调用返回值是用于标识管道文件的文件描述符 filedes，只有调用 pipe(filedes)的进程及其子进程才能识别该文件描述符，并利用管道文件进行通信。通过无名管道进行通信要注意读写端需要通过 lock 锁对管道的访问进行互斥控制。

（2）用于任意两个进程间通信的命名管道 named pipe。命名管道通过 mknod(const char *path, mode_t mod, dev_t dev)、mkfifo(const char *path, mode_t mode)创建，是一个具有路径名、在文件系统中长期存在的特殊类型的 FIFO 文件，读写遵循先进先出（first in first out），管道文件读是从文件开始处返回数据，对管道的写是将数据添加到管道末尾。命名管道的名字存在于文件系统中，内容存放在内存中。访问命名管道前，需要使用open()打开该文件。管道是单向的、半双工的，数据只能向一个方向流动，双向通信时需要建立两个管道。一个命名管道的所有实例共享同一个路径名，但是每一个实例均拥有独立的缓存与句柄。只要可以访问正确的与之关联的路径，进程间就能够彼此相互通信。

要求：编程实现进程间的（无名）管道、命名管道通信，方式如下。

(1) 管道通信：

父进程使用 `fork()` 系统调用创建子进程；

子进程作为写入端，首先将管道利用 `lockf()` 加锁，然后向管道中写入数据，并解锁管道；

父进程作为读出端，从管道中读出子进程写入的数据。

(2) 命名管道通信：

读者进程使用 `mknod` 或 `mkfifo` 创建命名管道；

读者进程、写者进程使用 `open()` 打开创建的管道文件；

读者进程、写者进程分别使用 `read()`、`write()` 读写命名管道文件中的内容；

通信完毕，读者进程、写者进程使用 `close` 关闭命名管道文件。

3.4 线程同步与互斥

该部分实验有三道题目可供实现，此处选择题目三。

题目 3. 医院门诊

校医院口腔科每天向患者提供 $N=30$ 个挂号就诊名额。患者到达医院后，如果有号，则挂号，并在候诊室排队等待就医；如果号已满，则离开医院。在诊疗室内，有 $M=3$ 位医生为患者提供治疗服务。如果候诊室有患者等待并且诊疗室内有医生处于“休息”态，则从诊疗室挑选一位患者，安排一位医生为其治疗，医生转入“工作”状态；如果三位医生均处于“工作”态，候诊室内患者需等待。当无患者候诊时，医生转入“休息”状态。

要求：

- (1) 采用信号量机制，描述患者、医生的行为；
- (2) 设置医生忙闲状态向量 $DState[M]$ ，记录每位医生的“工作”、“休息”状态；
- (3) 设置患者就诊状态向量 $PState[N]$ ，记录挂号成功后的患者的“候诊”、“就医”状态

实现原理：

本问题类似于睡眠理发师问题，既有多个进程互斥竞争使用资源，又有进程间同步。但与之不同之处在于：

- (1) 睡眠理发师问题中只有一位理发师，本问题中有多位医生为患者服务；
- (2) 引入两个作为共享数据结构的状态向量 $DState[M]$ 、 $PState[N]$ 。多个医生对 $DState[3]$ 的访问必须互斥，多个患者对的访问必须互斥也进行互斥，为此，要引入两个互斥信号量；
- (3) 医院每天只放 $N=30$ 个号，最先挂号的前 N 个患者可以获得号，并进入候诊室等待治疗。之后的患者由于拿不到当天号，没有必要等待，可以离开；而在睡眠理发师问题中，只要等候室有空位，顾客就可以进入等候室，等待理发服务。

3.5 多核多线程性能分析

- 实验 5.1 观察实验平台物理 cpu、CPU 核和逻辑 cpu 的数目
- 实验 5.2 单线程/进程串行 vs 两线程并行 vs 多线程加锁并行程序对比
- 实验 5.3 多线程加锁 vs 多线程不加锁 对比
- 实验 5.4 针对 Cache 的优化
- 实验 5.5 CPU 亲和力对并行程序影响

- 八种实现方案运行时间对比总结

4. 实验设计原理

4.1 进程管理模块设计原理

4.1.1 进程创建机制

使用 `fork()` 系统调用创建子进程，该调用通过复制父进程的地址空间创建一个新进程。父子进程在 `fork()` 返回后开始并发执行，通过返回值区分父子进程（子进程返回0，父进程返回子进程PID）。

4.1.2 进程程序加载

通过 `exec()` 系列函数替换进程的内存映像，加载新的可执行程序。实验中采用 `execvp()` 函数，它会在PATH环境变量指定的目录中搜索可执行文件，并允许传递参数列表。

4.1.3 进程控制机制

- 使用 `wait()` 系统调用使父进程阻塞等待子进程终止
- 通过 `kill()` 发送信号控制进程行为
- 利用 `WIFEXITED` 等宏检查子进程终止状态

4.1.4 进程并发原理

操作系统通过进程调度器在CPU上切换执行多个进程，创造并发执行的假象。父子进程的执行顺序取决于调度策略，可能产生不同的交织结果。

4.1.5 主要流程

先由主进程创建两个子进程，子进程1通过 `execvp()` 函数调用外程序，子进程2循环5次打印字符串模拟运行流程，运行到中间时主进程发送终止信号终止子进程2。

4.2 线程管理模块设计原理

4.2.1 线程创建机制

使用 `pthread_create()` 函数创建POSIX线程，与进程创建不同，线程共享同一进程的地址空间，创建开销显著小于进程。

4.2.2 线程终止方式比较

- **return返回**：线程函数正常返回，返回值传递给 `pthread_join()`
- **pthread_exit()**：显式终止线程，可指定退出值
- **pthread_cancel()**：异步取消线程，被取消线程返回 `PTHREAD_CANCELED`

4.2.3 线程同步机制

通过 `pthread_join()` 实现线程同步，主线程阻塞等待指定线程终止，并获取其返回值。此机制确保线程执行顺序的可控性。

4.2.4 主要流程

主线程首先创建三个线程分别执行不同的任务：线程1循环打印后通过return正常返回，线程2循环打印后通过pthread_exit显式退出，线程3设置延迟取消并循环检查取消点。创建完成后主线程立即取消线程3，随后等待所有线程结束。线程3在检查取消点时响应取消请求而终止，线程1和线程2正常完成各自循环。最后主线程通过pthread_join收集线程返回值，其中线程3因被取消而返回PTHREAD_CANCELED，完整演示了三种不同的线程终止方式。

4.3 进程通信模块设计原理

4.3.1 无名管道通信原理

- 通过 pipe() 系统调用创建，返回两个文件描述符（读端和写端）
- 基于内核缓冲区实现单向数据流
- 只能用于具有亲缘关系的进程间通信
- 生命周期随进程结束而终止

4.3.2 命名管道通信原理

- 通过 mkfifo() 创建具有名称的特殊文件
- 在文件系统中可见，通过路径名访问
- 允许无亲缘关系的进程通信
- 独立于进程存在，具有持久性

4.3.3 管道同步控制

使用 lockf() 对管道文件描述符加锁，实现读写进程间的同步，防止多个进程同时写入导致数据混乱，确保数据的完整性和一致性。

4.3.4 主要流程

主进程首先演示命名管道通信：创建FIFO文件后，fork出写者子进程和读者子进程，写者循环发送四条消息到命名管道，读者从管道读取并打印消息。主进程等待两个子进程结束后删除FIFO文件。接着主进程演示无名管道通信：通过pipe创建无名管道后fork出子进程，子进程向管道写入两条消息，父进程从管道读取消息。子进程通过lockf确保写入操作的原子性，父进程等待读取完成后结束程序。

4.4 线程通信模块设计原理

4.4.1 互斥锁原理

- 通过 pthread_mutex_t 类型声明互斥锁
- 使用 pthread_mutex_lock() 和 pthread_mutex_unlock() 保护临界区
- 确保同一时刻只有一个线程访问共享资源
- 防止数据竞争导致的不一致性问题

4.4.2 共享内存访问模式

多个线程并发访问同一结构体变量，主线程和子线程都可能修改共享数据。通过互斥锁序列化对共享数据的访问，保证操作的原子性。

4.4.3 临界区设计原则

临界区应尽可能小，只包含必须同步的共享数据访问操作，减少锁的持有时间，提高程序并发性能。

4.4.4 主要流程

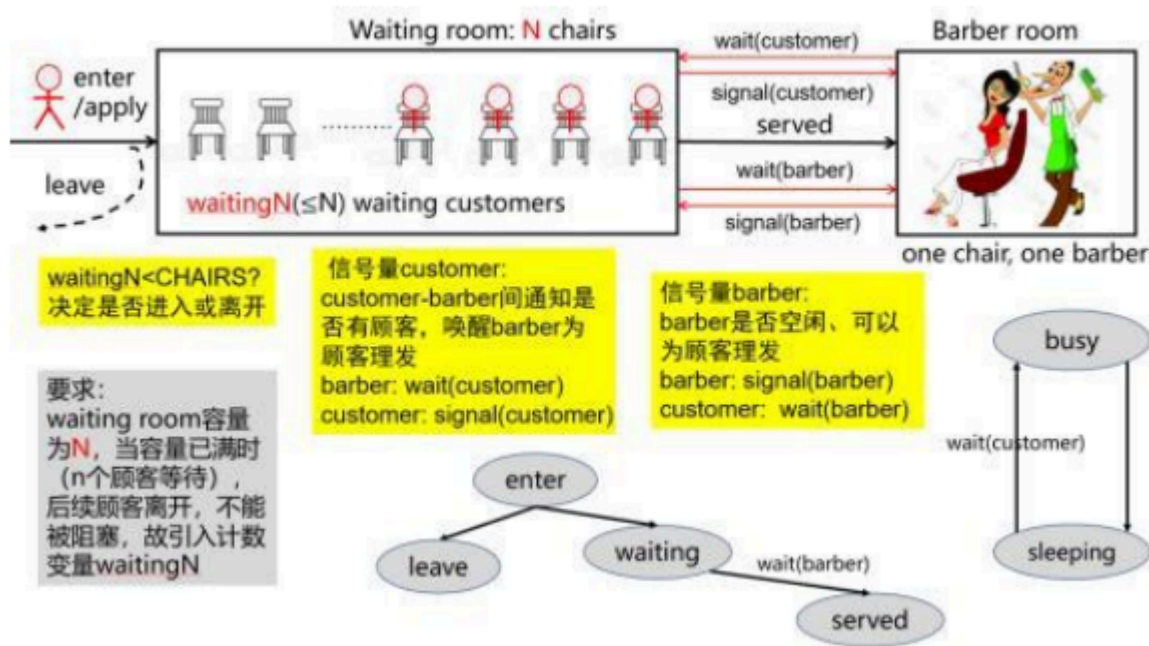
主线程初始化共享数据结构（包含计数器和消息），创建三个线程：一个写入线程循环3次增加计数器并修改消息，两个读取线程循环4次读取并打印共享数据。主线程在创建子线程后也获取互斥锁修改共享数据（计数器翻倍并写入主线程消息）。所有线程通过互斥锁保护对共享数据的访问。最后主线程等待所有子线程结束后，输出最终共享数据并销毁互斥锁。

4.5 线程同步与互斥模块设计原理

4.5.1 生产者-消费者模型应用

本问题类似于睡眠理发师问题，既有多个进程互斥竞争使用资源，又有进程间同步。但与之不同之处在于：

- (1) 睡眠理发师问题中只有一位理发师，本问题中有多位医生为患者服务；
- (2) 引入两个作为共享数据结构的状态向量 $DState[M]$ 、 $PState[N]$ 。多个医生对 $DState[3]$ 的访问必须互斥，多个患者对的访问必须互斥也进行互斥，为此，需要引入两个互斥信号量；
- (3) 医院每天只放 $N=30$ 个号，最先挂号的前 N 个患者可以获得号，并进入候诊室等待治疗。之后的患者由于拿不到当天号，没有必要等待，可以离开；而在睡眠理发师问题中，只要等候室有空位，顾客就可以进入等候室，等待理发服务。



4.5.2 条件变量同步机制

- 使用 `pthread_cond_t` 定义条件变量
- 通过 `pthread_cond_wait()` 使线程在条件不满足时阻塞
- 通过 `pthread_cond_signal()` 唤醒等待条件的线程
- 避免忙等待，提高CPU利用率

4.5.3 状态变量设计

维护两个状态向量：`Pstate` 记录患者状态（未挂号、候诊、已就诊），`Dstate` 记录医生状态（空闲、忙碌）。通过状态变量实现精确的同步控制。

4.5.4 双重互斥锁设计

使用两个独立的互斥锁分别保护患者相关资源和医生相关资源，减少锁竞争，提高系统并发度。

4.5.5 主要流程

主线程创建M个医生线程和N+5个患者线程模拟医院门诊。患者线程尝试挂号，若挂号数未达上限则进入候诊队列并唤醒等待的医生线程。医生线程循环检查候诊队列，若有患者且自身空闲则进行治疗，治疗期间更新患者状态和医生状态。治疗完成后释放医生资源并唤醒其他等待的医生线程。主线程先等待所有患者线程结束，再通过条件变量广播唤醒所有医生线程，最后等待所有医生线程结束。程序结束时输出总治疗人数和剩余候诊人数。

4.6 多核多线程性能分析设计原理

4.6.1 CPU信息检测原理

- 通过解析 `/proc/cpuinfo` 文件获取CPU核心数、型号等信息
- 通过 `sysconf(_SC_NPROCESSORS_ONLN)` 获取在线CPU数量
- 为后续的CPU亲和性设置提供基础信息

4.6.2 并行加速比分析原理

- 阿姆达尔定律： $S=(1-p)+np$
- 其中p为可并行部分比例，n为处理器核心数
- 通过比较单线程与多线程执行时间，计算实际加速比
- 分析并行化效率，识别性能瓶颈

4.6.3 锁开销分析原理

- 互斥锁引入的额外开销包括：锁获取/释放操作、线程阻塞/唤醒上下文切换
- 测试不同锁粒度对性能的影响
- 分析锁竞争导致的性能下降比例
- 评估无锁数据结构的潜在优势

4.6.4 Cache优化原理

- **Cache行对齐**：将频繁访问的数据对齐到Cache行边界，减少伪共享
- **数据局部性优化**：重新组织数据布局，提高空间局部性和时间局部性
- **循环分块**：将大循环分解为适合Cache大小的块
- 通过对比优化前后性能，验证Cache优化效果

4.6.5 CPU亲和性原理

- 通过 `sched_setaffinity()` 将线程绑定到特定CPU核心
- 减少线程在核心间的迁移开销
- 提高Cache命中率，减少一致性协议开销
- 特别适用于计算密集型、数据局部性好的应用

4.6.6 性能测试方法学

- 多次运行取平均值，减少测量误差

- 采用K-best测量方法，取5次最佳进行计算
- 使用高精度计时器（如 `clock_gettime()`）
- 排除系统波动影响，确保结果可靠性
- 对比不同场景下的性能表现，得出有意义的结论

5. 实验步骤

5.1 环境准备

- 配置Linux双系统开发环境
- 安装必要的开发工具和库
- 安装Pthread库并验证Pthread线程库可用性

ubuntu 下默认是没有pthread库的 即使在编译的时候 加上 `-lpthread` 也不行
man不到相关函数
使用下面的指令安装 就可以了

```
sudo apt-get install glibc-doc  
sudo apt-get install manpages-posix-dev
```

然后在用 `man -k pthread_create` 就可以找到了

5.2 第一组 进程的创建和管理

实验1-1：阅读 Linux 内核源码，分析 Linux 进程的组成，了解进程状态，观察进PCB/task_struct 等进程管理数据结构

实验1-2：利用Linux 内核提供的 fork()、exec()、wait()、exit()、kill()等系统调用，创建管理多个进程，观察父子进程的结构和并发行为，掌握睡眠、撤销等进程控制方法；

实验1-3：了解 ps、top、pstree -h、vmstat、strace、ltrace、sleep x、kill、jobs 等命令的功能，使用这些命令观察进程结构和行为；

5.3 第二组 线程的创建和管理

实验2-1：了解POSIX线程标准库（Pthread线程库）相关知识，分析Pthread线程结构，掌握所提供的线程管理API，如pthread_create、pthread_join、pthread_self、pthread_detach、pthread_exit等函数的使用方法。

实验2-2：参照线程创建及管理程序示例，查阅参考资料，利用Pthread API创建和管理线程，观察线程的结构和并发执行行为。要求至少使用pthread_create、pthread_exit、pthread_join、pthread_self等四个API函数。

实验2-3：在线程代码中不同位置添加随机延迟（使用sleep()函数），使线程执行横跨多个时间片，保证线程执行时间不少于3个时间片，观察线程的并发行为特征。

5.4 第三组 进程间通信

实验3-1：学习Linux系统提供的四种进程间通信机制的系统调用和库函数使用方法，重点掌握管道通信的实现原理。

实验3-2：编程实现无名管道通信，父进程创建子进程后，子进程作为写入端向管道写入数据，父进程作为读出端从管道读取数据，使用lockf()函数实现管道访问的互斥控制。

实验3-3：编程实现命名管道通信，创建FIFO特殊文件，实现读者进程和写者进程通过命名管道进行数据交换，掌握open()、read()、write()、close()等文件操作函数在管道通信中的应用。

5.5 第四组 线程间通信

实验4-1：了解Linux线程概念和线程间通信机制，掌握多线程编程的基本方法。

实验4-2：参照线程间参数传递示例程序，编程实现线程间通过共享数据类型（整型变量、字符串、结构体等）进行信息传递。

实验4-3：验证同一进程内多个线程共享地址空间的特性，观察线程间通过共享全局变量进行快速通信的效果。

5.6 第五组 同步与互斥

实验5-1：针对医院门诊同步问题，设计基于信号量的正确解决方案，分析医生资源分配和患者就诊流程的同步需求。

实验5-2：利用Pthread提供的pthread_mutex_init()、pthread_mutex_lock()、pthread_mutex_unlock()等线程同步互斥API，以线程方式编程实现医院门诊问题的解决方案。

实验5-3：观察程序运行结果，重点分析信号量设计方案是否合理，验证程序运行是否符合预期要求，确保不会发生死锁或不符合题目要求的情况。

5.7 第六组 多核多线程编程及性能分析

实验6-1：观察实验平台物理CPU、CPU核和逻辑CPU的数目，了解硬件环境对多线程程序性能的影响。

实验6-2：编写单线程/进程串行程序、两线程并行程序和三线程加锁并行程序，对比分析不同并行化方案对程序运行时间的影响。

实验6-3：对比三线程加锁与三线程不加锁程序的性能差异，分析锁机制对多线程程序执行效率的影响。

实验6-4：针对Cache进行优化，通过数据对齐和布局优化减少Cache伪共享，提高程序性能。

实验6-5：研究CPU亲和力和对并行程序的影响，通过线程绑定技术优化线程在多个CPU核心间的调度。

实验6-6：对比分析八种不同实现方案的运行时间，以图表形式展示性能测试结果，总结多核多线程编程的最佳实践。

5.8 运行及测试验证

- 编写run.sh文件，运行方法：`./run.sh`，一键编译和运行所有代码文件，run.sh文件具体内容见附录
- 验证同步机制的正确性
- 收集性能测试数据

5.9 性能分析

- 对比不同实现方案的运行时间
- 分析锁机制、Cache优化、CPU亲和性的影响
- 绘制性能对比图表

6. 实验结果及分析

6.1 进程管理实验结果

实验 1-1:

1. Linux进程组成分析

进程主要组成部分:

- **进程控制块(PCB)**: task_struct结构体
- **程序段**: 要执行的程序代码
- **数据段**: 程序运行时的数据
- **堆栈段**: 运行时的堆栈信息

2. task_struct

```
struct task_struct {
    volatile long state;           // 进程状态
    void *stack;                   // 进程堆栈指针
    int prio;                       // 动态优先级
    int static_prio;               // 静态优先级
    struct list_head tasks;        // 进程链表
    struct mm_struct *mm;          // 内存管理信息
    struct files_struct *files;    // 打开文件信息
    pid_t pid;                     // 进程ID
    pid_t tgid;                    // 线程组ID
    struct task_struct *real_parent; // 真实父进程
    struct task_struct *parent;    // 父进程
    struct list_head children;     // 子进程链表
    struct list_head sibling;       // 兄弟进程链表
    struct task_struct *group_leader; // 线程组领导
    char comm[TASK_COMM_LEN];     // 可执行程序名称
    // ... 更多字段
};
```

3. 进程状态

TASK_RUNNING (0): 可运行状态
TASK_INTERRUPTIBLE (1): 可中断睡眠状态
TASK_UNINTERRUPTIBLE (2): 不可中断睡眠状态
__TASK_STOPPED (4): 停止状态
__TASK_TRACED (8): 被跟踪状态
EXIT_ZOMBIE (16): 僵尸状态
EXIT_DEAD (32): 死亡状态

实验1-2:

运行结果:

```
实验1 - 进程创建与管理:
父进程PID: 16303
子进程1 PID: 16304, 父进程PID: 16303
子进程1调用exec执行ls -l命令:
父进程创建了两个子进程: 16304 和 16305
子进程2 PID: 16305
子进程2正在工作...0
总计 252
-rwxrwxr-x 1 ji ji 16432 12月 17 23:04 lab1
-rw-rw-r-- 1 ji ji 2789 12月 16 21:09 lab1.c
-rwxrwxr-x 1 ji ji 16568 12月 17 23:04 lab2
-rw-rw-r-- 1 ji ji 2660 12月 16 21:00 lab2.c
-rwxrwxr-x 1 ji ji 16904 12月 17 23:04 lab3
-rw-rw-r-- 1 ji ji 3966 12月 16 20:13 lab3.c
-rwxrwxr-x 1 ji ji 16640 12月 17 23:04 lab4
-rw-rw-r-- 1 ji ji 2595 12月 16 20:50 lab4.c
-rwxrwxr-x 1 ji ji 16952 12月 17 23:04 lab5
-rw-rw-r-- 1 ji ji 3588 12月 16 22:57 lab5.c
-rwxrwxr-x 1 ji ji 16216 12月 17 23:04 lab6-1
-rw-rw-r-- 1 ji ji 1022 12月 17 22:33 lab6-1.c
-rwxrwxr-x 1 ji ji 16832 12月 17 23:04 lab6-2
-rw-rw-r-- 1 ji ji 4265 12月 17 13:39 lab6-2.c
-rwxrwxr-x 1 ji ji 16840 12月 17 23:04 lab6-3
-rw-rw-r-- 1 ji ji 4013 12月 17 13:39 lab6-3.c
-rwxrwxr-x 1 ji ji 16696 12月 17 23:04 lab6-4
-rw-rw-r-- 1 ji ji 6640 12月 17 13:48 lab6-4.c
-rwxrwxr-x 1 ji ji 16800 12月 17 23:04 lab6-5
-rw-rw-r-- 1 ji ji 4391 12月 17 13:57 lab6-5.c
-rwxrwxr-x 1 ji ji 804 12月 17 21:28 run.sh
子进程2正在工作...1
子进程2正在工作...2
子进程2正在工作...3

父进程尝试终止子进程2 (PID: 16305)...
已向子进程2发送SIGTERM信号
子进程1 (PID: 16304) 已结束
子进程2 (PID: 16305) 被信号终止, 信号编号: 15
所有子进程已结束, 父进程退出
```

● **ji@ji:~/文档/codes/lab\$./lab1**

父进程PID: 16822

父进程创建了两个子进程: 16823 和 16824

子进程1 PID: 16823, 父进程PID: 16822

子进程1调用exec执行ls -l命令:

子进程2 PID: 16824

子进程2正在工作...0

总计 252

```
-rwxrwxr-x 1 ji ji 16432 12月 17 23:04 lab1
-rw-rw-r-- 1 ji ji 2789 12月 16 21:09 lab1.c
-rwxrwxr-x 1 ji ji 16568 12月 17 23:04 lab2
-rw-rw-r-- 1 ji ji 2660 12月 16 21:00 lab2.c
-rwxrwxr-x 1 ji ji 16904 12月 17 23:04 lab3
-rw-rw-r-- 1 ji ji 3966 12月 16 20:13 lab3.c
-rwxrwxr-x 1 ji ji 16640 12月 17 23:04 lab4
-rw-rw-r-- 1 ji ji 2595 12月 16 20:50 lab4.c
-rwxrwxr-x 1 ji ji 16952 12月 17 23:04 lab5
-rw-rw-r-- 1 ji ji 3588 12月 16 22:57 lab5.c
-rwxrwxr-x 1 ji ji 16216 12月 17 23:04 lab6-1
-rw-rw-r-- 1 ji ji 1022 12月 17 22:33 lab6-1.c
-rwxrwxr-x 1 ji ji 16832 12月 17 23:04 lab6-2
-rw-rw-r-- 1 ji ji 4265 12月 17 13:39 lab6-2.c
-rwxrwxr-x 1 ji ji 16840 12月 17 23:04 lab6-3
-rw-rw-r-- 1 ji ji 4013 12月 17 13:39 lab6-3.c
-rwxrwxr-x 1 ji ji 16696 12月 17 23:04 lab6-4
-rw-rw-r-- 1 ji ji 6640 12月 17 13:48 lab6-4.c
-rwxrwxr-x 1 ji ji 16800 12月 17 23:04 lab6-5
-rw-rw-r-- 1 ji ji 4391 12月 17 13:57 lab6-5.c
-rwxrwxr-x 1 ji ji 804 12月 17 21:28 run.sh
```

子进程2正在工作...1

子进程2正在工作...2

子进程2正在工作...3

父进程尝试终止子进程2 (PID: 16824)...

已向子进程2发送SIGTERM信号

子进程1 (PID: 16823) 已结束

子进程2 (PID: 16824) 被信号终止, 信号编号: 15

所有子进程已结束, 父进程退出

结果分析：

- 父子进程正确创建并并发执行
- exec()成功加载外部程序
- kill()能够正确终止指定进程

实验1-3：

命令	功能说明	常用参数示例	输出内容说明
ps	显示当前进程的快照信息	ps ps -aux ps -ef	显示进程ID(PID)、父进程ID(PPID)、终端、状态、CPU/内存占用等
top	动态实时监控系统和资源	top top -bn1	实时显示进程列表、系统负载、内存使用、CPU占用率等
pstree	以树状结构显示进程关系	pstree pstree -p	显示进程的父子关系，直观展示进程层次结构
vmstat	报告虚拟内存和系统统计信息	vmstat 1 5	显示内存、交换分区、IO、系统中断、CPU使用情况等统计
strace	跟踪进程的系统调用和信号	strace command strace -p PID	显示程序执行时的系统函数调用、参数传递、返回值等
ltrace	跟踪进程的库函数调用	ltrace command	显示程序调用的动态库函数及其参数（需要安装）
sleep	使进程暂停执行指定时间	sleep 10 sleep 2m	进程进入睡眠状态，常用于脚本延时或测试
kill	向进程发送信号	kill PID kill -9 PID kill -STOP PID	终止进程(-9强制)、暂停进程(STOP)、继续进程(CONT)等
jobs	显示当前shell的后台任务	jobs jobs -l	显示后台运行的作业列表及其状态（运行、停止等）

=== 进程管理命令快速演示 ===

1. ps命令演示：

当前进程：

```
PID TTY          TIME CMD
12720 pts/1        00:00:00 bash
19816 pts/1        00:00:00 lab1-3.sh
19817 pts/1        00:00:00 ps
```

2. top命令演示（快速显示）：

```
top - 00:07:44 up 3:59, 1 user, load average: 0.26, 0.38, 0.36
任务: 416 total, 1 running, 410 sleeping, 0 stopped, 5 zombie
%Cpu(s): 3.4 us, 1.1 sy, 0.0 ni, 95.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15722.4 total, 6837.2 free, 4556.9 used, 5613.5 buff/cache
```

MiB Swap: 9537.0 total, 9537.0 free, 0.0 used. 11165.5 avail Mem

进程号	USER	PR	NI	VIRT	RES	SHR	%CPU	%MEM	TIME+	COMMAND
19818	ji	20	0	14688	5744	3580 R	9.1	0.0	0:00.01	top
1	root	20	0	23696	14912	9528 S	0.0	0.1	0:02.58	systemd
2	root	20	0	0	0	0 S	0.0	0.0	0:00.02	kthreadd

3. pstree命令演示:

```
systemd-+-ModemManager---3*[{ModemManager}]
          |-NetworkManager---3*[{NetworkManager}]
          |-accounts-daemon---3*[{accounts-daemon}]
          |-avahi-daemon---avahi-daemon
          |-bluetoothd
```

4. vmstat命令演示:

```
procs -----memory----- --swap-- -----io---- -system-- -----cpu-----
 r  b  交换  空闲  缓冲  缓存    si    so    bi    bo    in    cs  us  sy  id  wa  st  gu
 0  0      0 7002972 154472 5596756    0    0   193   280 3787    4  2  1 97  0  0  0
 0  0      0 7006868 154472 5593996    0    0    0    0 2229 2353  0  0 99  0  0  0
```

5. 创建测试进程用于演示:

创建sleep进程, PID: 19826

6. jobs命令演示:

```
[1]+  运行中                sleep 60 &
```

7. kill命令演示:

终止测试进程...

8. strace命令演示:

```
test
% time      seconds  usecs/call     calls   errors syscall
-----
 21.81      0.000041         13         3         mprotect
 11.70      0.000022         22         1         munmap
```

9. ltrace命令演示:

```
getenv("POSIXLY_CORRECT")          = nil
strchr("echo", '/')                 = nil
setlocale(LC_ALL, "")                = "zh_CN.UTF-8"
bindtextdomain("coreutils", "/usr/share/locale") = "/usr/share/locale"
textdomain("coreutils")              = "coreutils"
```

10. sleep命令演示:

等待2秒...

等待结束


```
ji@ji:~/文档/codes/Lab$ ./lab1-3.sh
```

1. ps命令演示：

当前进程：

PID	TTY	TIME	CMD
12720	pts/1	00:00:00	bash
19816	pts/1	00:00:00	lab1-3.sh
19817	pts/1	00:00:00	ps

2. top命令演示（快速显示）：

```
top - 00:07:44 up 3:59, 1 user, load average: 0.26, 0.38, 0.36
任务: 416 total, 1 running, 410 sleeping, 0 stopped, 5 zombie
%Cpu(s): 3.4 us, 1.1 sy, 0.0 ni, 95.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15722.4 total, 6837.2 free, 4556.9 used, 5613.5 buff/cache
MiB Swap: 9537.0 total, 9537.0 free, 0.0 used. 11165.5 avail Mem
```

进程号	USER	PR	NI	VIRT	RES	SHR	%CPU	%MEM	TIME+	COMMAND
19818	ji	20	0	14688	5744	3580 R	9.1	0.0	0:00.01	top
1	root	20	0	23696	14912	9528 S	0.0	0.1	0:02.58	systemd
2	root	20	0	0	0	0 S	0.0	0.0	0:00.02	kthreadd

3. pstree命令演示：

```
systemd--+-ModemManager---3*[{ModemManager}]
          |-NetworkManager---3*[{NetworkManager}]
          |-accounts-daemon---3*[{accounts-daemon}]
          |-avahi-daemon---avahi-daemon
          |-bluetoothd
```

4. vmstat命令演示：

```
procs -----memory----- --swap-- -----io---- -system-- -----cpu-----
 r  b 交换 空闲 缓冲 缓存  si  so  bi  bo  in  cs us sy id wa st gu
0  0      0 7002972 154472 5596756  0  0 193 280 3787  4 2 1 97 0 0 0
0  0      0 7006868 154472 5593996  0  0  0  0 2229 2353 0 0 99 0 0 0
```

5. 创建测试进程用于演示：

创建sleep进程，PID：19826

6. jobs命令演示：

```
[1]+  运行中                sleep 60 &
```

7. kill命令演示：

终止测试进程...

8. strace命令演示：

```
test
% time      seconds  usecs/call     calls   errors syscall
-----
21.81      0.000041         13         3         mprotect
11.70      0.000022         22         1         munmap
```

9. ltrace命令演示：

```
getenv("POSIXLY_CORRECT")          = nil
strchr("echo", '/')                 = nil
setlocale(LC_ALL, "")                = "zh_CN.UTF-8"
bindtextdomain("coreutils", "/usr/share/locale") = "/usr/share/locale"
textdomain("coreutils")              = "coreutils"
```

10. sleep命令演示：

等待2秒...

等待结束

=== 演示结束 ===

6.2 线程管理实验结果

实验2-1：

Pthread线程库API函数功能说明表

函数名称	功能说明	参数说明	返回值	使用场景
pthread_create	创建新线程	thread: 线程标识符指针 attr: 线程属性(可为NULL) start_routine: 线程函数 arg: 传递给线程函数的参数	成功: 0 失败: 错误码	创建并发执行的线程
pthread_exit	终止当前线程	retval: 线程退出时的返回值	无返回值	线程正常退出, 释放资源
pthread_cancel	取消指定线程	thread: 要取消的线程ID	成功: 0 失败: 错误码	强制终止其他线程的执行
pthread_join	等待指定线程结束	thread: 要等待的线程ID retval: 存储线程返回值	成功: 0 失败: 错误码	线程同步, 资源回收
pthread_self	获取当前线程ID	无参数	当前线程的ID	线程自我标识
pthread_detach	分离线程 (自动回收)	thread: 要分离的线程ID	成功: 0 失败: 错误码	创建不需要等待的守护线程
pthread_equal	比较两个线程ID是否相等	t1, t2: 要比较的线程ID	相等: 非0 不相等: 0	线程身份验证
pthread_kill	向线程发送信号	thread: 目标线程ID sig: 信号值	成功: 0 失败: 错误码	线程间通信, 中断处理

互斥锁相关函数

函数名称	功能说明	参数说明	返回值	使用场景
pthread_mutex_init	初始化互斥锁	mutex: 互斥锁指针 attr: 属性(可为NULL)	成功: 0 失败: 错误码	创建互斥锁, 保护临界区
pthread_mutex_lock	加锁互斥锁	mutex: 互斥锁指针	成功: 0 失败: 错误码	进入临界区前的保护
pthread_mutex_unlock	解锁互斥锁	mutex: 互斥锁指针	成功: 0 失败: 错误码	离开临界区后的释放

函数名称	功能说明	参数说明	返回值	使用场景
pthread_mutex_trylock	尝试加锁 (非阻塞)	mutex: 互斥锁指针	成功: 0 失败: 错误码	避免死锁的非阻塞操作
pthread_mutex_destroy	销毁互斥锁	mutex: 互斥锁指针	成功: 0 失败: 错误码	清理互斥锁资源

条件变量相关函数

函数名称	功能说明	参数说明	返回值	使用场景
pthread_cond_init	初始化条件变量	cond: 条件变量指针 attr: 属性(可为NULL)	成功: 0 失败: 错误码	线程间同步通信
pthread_cond_wait	等待条件变量	cond: 条件变量 mutex: 互斥锁	成功: 0 失败: 错误码	线程等待特定条件满足
pthread_cond_signal	唤醒一个等待线程	cond: 条件变量指针	成功: 0 失败: 错误码	通知一个等待线程条件满足
pthread_cond_broadcast	唤醒所有等待线程	cond: 条件变量指针	成功: 0 失败: 错误码	通知所有等待线程条件满足
pthread_cond_destroy	销毁条件变量	cond: 条件变量指针	成功: 0 失败: 错误码	清理条件变量资源

线程清理函数

函数名称	功能说明	参数说明	返回值	使用场景
pthread_cleanup_push	注册线程清理函数	routine: 清理函数 arg: 清理函数参数	无返回值	确保线程退出时资源释放
pthread_cleanup_pop	执行/移除清理函数	execute: 是否执行清理函数	无返回值	与push配对使用，管理清理栈

实验2-2：

运行结果：

```
主线程ID：137787388090176
=== 开始创建线程 ===
线程1（ID：137787384788672）启动 - 使用return退出
线程1工作：0
=== 所有线程创建完成 ===
线程1 ID：137787384788672
线程2 ID：137787376395968
线程3 ID：137787368003264
```

```
主线程主动取消线程3...
线程3 (ID: 137787368003264) 启动 - 将被主线程取消
线程3工作: 0 (线程ID: 137787368003264)
线程2 (ID: 137787376395968) 启动 - 使用pthread_exit退出
线程2工作: 0
线程2工作: 1
线程1工作: 1
线程2工作: 2
线程1工作: 2
线程2通过pthread_exit退出
线程1通过return退出
线程1返回值: 100
线程2返回值: 400
线程3: 被主线程取消终止 (PTHREAD_CANCELED)

=== 所有线程执行完成 ===
主线程结束
```

```

● ji@ji:~/文档/codes/lab$ ./lab2
主线程ID: 137787388090176
=== 开始创建线程 ===
线程1 (ID: 137787384788672) 启动 - 使用return退出
线程1工作: 0
=== 所有线程创建完成 ===
线程1 ID: 137787384788672
线程2 ID: 137787376395968
线程3 ID: 137787368003264

主线程主动取消线程3...
线程3 (ID: 137787368003264) 启动 - 将被主线程取消
线程3工作: 0 (线程ID: 137787368003264)
线程2 (ID: 137787376395968) 启动 - 使用pthread_exit退出
线程2工作: 0
线程2工作: 1
线程1工作: 1
线程2工作: 2
线程1工作: 2
线程2通过pthread_exit退出
线程1通过return退出
线程1返回值: 100
线程2返回值: 400
线程3: 被主线程取消终止 (PTHREAD_CANCELED)

=== 所有线程执行完成 ===
主线程结束

```

结果分析:

- 三种线程终止方式均正确实现
- 线程ID唯一标识每个线程
- pthread_join()正确获取线程返回值

6.3 进程通信实验结果

管道通信结果:

```

=== 命名管道通信示例 ===
命名管道创建成功: /tmp/myfifo
写者进程启动 (PID: 17067)
读者进程启动 (PID: 17068)
写者发送: 第一条消息: Hello from writer!
读者接收: 第一条消息: Hello from writer!

```

写者发送：第二条消息：命名管道通信测试

读者接收：第二条消息：命名管道通信测试

写者发送：第三条消息：进程间通信成功！

读者接收：第三条消息：进程间通信成功！

写者发送：END

读者接收：END

读者进程结束

写者进程结束

命名管道已删除

主进程结束

=== 无名管道通信示例 ===

父进程PID：17066

管道创建成功：读端=3，写端=4

父进程等待子进程数据...

子进程启动 (PID：17103)

子进程写入数据：Hello from child process! 这是子进程通过管道发送的消息。

父进程接收到数据：Hello from child process! 这是子进程通过管道发送的消息。

接收字节数：72

子进程写入第二条数据：这是子进程的第二条消息！

子进程结束

父进程接收到数据：这是子进程的第二条消息！

接收字节数：37

父进程结束

```

● ji@ji:~/文档/codes/lab$ ./lab3
=== 命名管道通信示例 ===
命名管道创建成功: /tmp/myfifo
写者进程启动 (PID: 17067)
读者进程启动 (PID: 17068)
写者发送: 第一条消息: Hello from writer!
读者接收: 第一条消息: Hello from writer!
写者发送: 第二条消息: 命名管道通信测试
读者接收: 第二条消息: 命名管道通信测试
写者发送: 第三条消息: 进程间通信成功!
读者接收: 第三条消息: 进程间通信成功!
写者发送: END
读者接收: END
读者进程结束
写者进程结束
命名管道已删除
主进程结束
=== 无名管道通信示例 ===
父进程PID: 17066
管道创建成功: 读端=3, 写端=4
父进程等待子进程数据...
子进程启动 (PID: 17103)
子进程写入数据: Hello from child process! 这是子进程通过管道发送的消息。
父进程接收到数据: Hello from child process! 这是子进程通过管道发送的消息。
接收字节数: 72
子进程写入第二条数据: 这是子进程的第二条消息!
子进程结束
父进程接收到数据: 这是子进程的第二条消息!
接收字节数: 37
父进程结束

```

结果分析:

- 命名管道和无名管道均正常工作
- 进程间消息正确传递
- 锁机制保证数据完整性

6.4 线程通信实验结果

线程通信结果:

```

主线程启动
初始共享数据: 计数=0, 消息=初始消息
线程1写入: 消息来自线程1, 计数: 1
线程2读取: 计数=1, 消息=消息来自线程1, 计数: 1
主线程17230写入: 消息来自主线程17230, 计数: 2
线程3读取: 计数=2, 消息=消息来自主线程17230, 计数: 2
线程1写入: 消息来自线程1, 计数: 3
线程2读取: 计数=3, 消息=消息来自线程1, 计数: 3
线程3读取: 计数=3, 消息=消息来自线程1, 计数: 3
线程1写入: 消息来自线程1, 计数: 4
线程2读取: 计数=4, 消息=消息来自线程1, 计数: 4
线程3读取: 计数=4, 消息=消息来自线程1, 计数: 4

```

```
线程2读取：计数=4，消息=消息来自线程1，计数：4
线程3读取：计数=4，消息=消息来自线程1，计数：4
最终共享数据：计数=4，消息=消息来自线程1，计数：4
主线程结束
```

```
● ji@ji:~/文档/codes/lab$ ./lab4
主线程启动
初始共享数据：计数=0，消息=初始消息
线程1写入：消息来自线程1，计数：1
线程2读取：计数=1，消息=消息来自线程1，计数：1
主线程17230写入：消息来自主线程17230，计数：2
线程3读取：计数=2，消息=消息来自主线程17230，计数：2
线程1写入：消息来自线程1，计数：3
线程2读取：计数=3，消息=消息来自线程1，计数：3
线程3读取：计数=3，消息=消息来自线程1，计数：3
线程1写入：消息来自线程1，计数：4
线程2读取：计数=4，消息=消息来自线程1，计数：4
线程3读取：计数=4，消息=消息来自线程1，计数：4
线程2读取：计数=4，消息=消息来自线程1，计数：4
线程3读取：计数=4，消息=消息来自线程1，计数：4
最终共享数据：计数=4，消息=消息来自线程1，计数：4
主线程结束
```

结果分析：

1. 数据一致性得到保障

在整个运行过程中，共享数据（计数器和消息）始终保持逻辑一致。例如：

当 计数=1 时，对应的消息是 消息来自线程1，计数：1。

当 计数=4 时，对应的消息是 消息来自线程1，计数：4。

这证明了互斥锁有效防止了“写操作”和“读操作”的并发执行，避免了线程在修改数据的过程中（例如，刚更新完计数器但还未更新消息），其他线程来读取到不一致的中间状态。

2. 临界区互斥访问生效

输出结果显示了严格的串行化访问顺序，没有出现两个线程的写入或读写操作交织在一起的情况。例如：

- 线程1写入 操作完成后，才有 线程2读取。
- 主线程写入 操作完成后，才有 线程3读取。

这表明 `pthread_mutex_lock` 和 `pthread_mutex_unlock` 成功地将对共享数据的访问变成了临界区，任一时刻最多只有一个线程在执行临界区代码，实现了互斥。

3. 线程执行顺序的不确定性

尽管访问是互斥的，但“哪个线程先获得锁”的顺序是由操作系统调度器决定的，具有一定的随机性。本次运行中观察到的顺序是：

线程1 -> 线程2 -> 主线程 -> 线程3 -> 线程1 ...

这体现了线程并发执行的特征：虽然每个临界区的操作是原子的，但线程之间的相对执行顺序是无法预测的。

对实验目的的验证

- **成功实现通信：**线程1和主线程通过修改共享数据成功向读取线程传递了信息。
- **成功实现同步：**互斥锁机制保证了通信过程的有序和数据正确，没有发生数据竞争（Data Race）。

6.5 同步互斥实验结果

医院门诊模拟结果：

患者 0：挂号成功，候诊中（当前候诊人数：1）
医生 2：开始治疗患者 0（已治疗：1/30）
患者 1：挂号成功，候诊中（当前候诊人数：1）
医生 1：开始治疗患者 1（已治疗：2/30）
患者 2：挂号成功，候诊中（当前候诊人数：1）
医生 0：开始治疗患者 2（已治疗：3/30）
患者 3：挂号成功，候诊中（当前候诊人数：1）
医生 2：完成治疗患者 0
患者 4：挂号成功，候诊中（当前候诊人数：2）
医生 1：完成治疗患者 1
患者 5：挂号成功，候诊中（当前候诊人数：3）
医生 2：开始治疗患者 3（已治疗：4/30）
医生 0：完成治疗患者 2
患者 6：挂号成功，候诊中（当前候诊人数：3）
医生 1：开始治疗患者 4（已治疗：5/30）
患者 7：挂号成功，候诊中（当前候诊人数：3）
医生 0：开始治疗患者 5（已治疗：6/30）
患者 8：挂号成功，候诊中（当前候诊人数：3）
患者 9：挂号成功，候诊中（当前候诊人数：4）
医生 2：完成治疗患者 3
患者 10：挂号成功，候诊中（当前候诊人数：5）
医生 1：完成治疗患者 4
患者 11：挂号成功，候诊中（当前候诊人数：6）
医生 2：开始治疗患者 6（已治疗：7/30）
医生 0：完成治疗患者 5
患者 12：挂号成功，候诊中（当前候诊人数：6）
医生 1：开始治疗患者 7（已治疗：8/30）
患者 13：挂号成功，候诊中（当前候诊人数：6）
医生 0：开始治疗患者 8（已治疗：9/30）
患者 14：挂号成功，候诊中（当前候诊人数：6）
患者 15：挂号成功，候诊中（当前候诊人数：7）
医生 2：完成治疗患者 6
患者 16：挂号成功，候诊中（当前候诊人数：8）
医生 1：完成治疗患者 7
患者 17：挂号成功，候诊中（当前候诊人数：9）
医生 2：开始治疗患者 9（已治疗：10/30）
医生 0：完成治疗患者 8

患者 18: 挂号成功, 候诊中 (当前候诊人数: 9)
医生 1: 开始治疗患者 10 (已治疗: 11/30)
患者 19: 挂号成功, 候诊中 (当前候诊人数: 9)
医生 0: 开始治疗患者 11 (已治疗: 12/30)
患者 20: 挂号成功, 候诊中 (当前候诊人数: 9)
患者 21: 挂号成功, 候诊中 (当前候诊人数: 10)
医生 2: 完成治疗患者 9
患者 22: 挂号成功, 候诊中 (当前候诊人数: 11)
医生 1: 完成治疗患者 10
患者 23: 挂号成功, 候诊中 (当前候诊人数: 12)
医生 2: 开始治疗患者 12 (已治疗: 13/30)
医生 0: 完成治疗患者 11
患者 24: 挂号成功, 候诊中 (当前候诊人数: 12)
医生 1: 开始治疗患者 13 (已治疗: 14/30)
患者 25: 挂号成功, 候诊中 (当前候诊人数: 12)
医生 0: 开始治疗患者 14 (已治疗: 15/30)
患者 26: 挂号成功, 候诊中 (当前候诊人数: 12)
患者 27: 挂号成功, 候诊中 (当前候诊人数: 13)
医生 2: 完成治疗患者 12
患者 28: 挂号成功, 候诊中 (当前候诊人数: 14)
医生 1: 完成治疗患者 13
患者 29: 挂号成功, 候诊中 (当前候诊人数: 15)
医生 2: 开始治疗患者 15 (已治疗: 16/30)
医生 0: 完成治疗患者 14
患者 30: 号已满, 离开医院
医生 1: 开始治疗患者 16 (已治疗: 17/30)
患者 31: 号已满, 离开医院
医生 0: 开始治疗患者 17 (已治疗: 18/30)
患者 32: 号已满, 离开医院
患者 33: 号已满, 离开医院
医生 2: 完成治疗患者 15
患者 34: 号已满, 离开医院
医生 1: 完成治疗患者 16
医生 2: 开始治疗患者 18 (已治疗: 19/30)
医生 0: 完成治疗患者 17
医生 1: 开始治疗患者 19 (已治疗: 20/30)
医生 0: 开始治疗患者 20 (已治疗: 21/30)
医生 2: 完成治疗患者 18
医生 1: 完成治疗患者 19
医生 2: 开始治疗患者 21 (已治疗: 22/30)
医生 0: 完成治疗患者 20
医生 1: 开始治疗患者 22 (已治疗: 23/30)
医生 0: 开始治疗患者 23 (已治疗: 24/30)
医生 2: 完成治疗患者 21
医生 1: 完成治疗患者 22
医生 2: 开始治疗患者 24 (已治疗: 25/30)
医生 0: 完成治疗患者 23
医生 1: 开始治疗患者 25 (已治疗: 26/30)
医生 0: 开始治疗患者 26 (已治疗: 27/30)
医生 2: 完成治疗患者 24
医生 1: 完成治疗患者 25
医生 2: 开始治疗患者 27 (已治疗: 28/30)
医生 0: 完成治疗患者 26

医生 1: 开始治疗患者 28 (已治疗: 29/30)
医生 0: 开始治疗患者 29 (已治疗: 30/30)
医生 2: 完成治疗患者 27
医生 1: 完成治疗患者 28
医生 0: 完成治疗患者 29

=== 今日门诊结束 ===
总治疗患者: 30/30
剩余候诊患者: 0

● `ji@ji:~/文档/codes/lab$ ./lab5`

患者 0: 挂号成功, 候诊中 (当前候诊人数: 1)
医生 2: 开始治疗患者 0 (已治疗: 1/30)
患者 1: 挂号成功, 候诊中 (当前候诊人数: 1)
医生 1: 开始治疗患者 1 (已治疗: 2/30)
患者 2: 挂号成功, 候诊中 (当前候诊人数: 1)
医生 0: 开始治疗患者 2 (已治疗: 3/30)
患者 3: 挂号成功, 候诊中 (当前候诊人数: 1)
医生 2: 完成治疗患者 0
患者 4: 挂号成功, 候诊中 (当前候诊人数: 2)
医生 1: 完成治疗患者 1
患者 5: 挂号成功, 候诊中 (当前候诊人数: 3)
医生 2: 开始治疗患者 3 (已治疗: 4/30)
医生 0: 完成治疗患者 2
患者 6: 挂号成功, 候诊中 (当前候诊人数: 3)
医生 1: 开始治疗患者 4 (已治疗: 5/30)
患者 7: 挂号成功, 候诊中 (当前候诊人数: 3)
医生 0: 开始治疗患者 5 (已治疗: 6/30)
患者 8: 挂号成功, 候诊中 (当前候诊人数: 3)
患者 9: 挂号成功, 候诊中 (当前候诊人数: 4)
医生 2: 完成治疗患者 3
患者 10: 挂号成功, 候诊中 (当前候诊人数: 5)
医生 1: 完成治疗患者 4
患者 11: 挂号成功, 候诊中 (当前候诊人数: 6)
医生 2: 开始治疗患者 6 (已治疗: 7/30)
医生 0: 完成治疗患者 5
患者 12: 挂号成功, 候诊中 (当前候诊人数: 6)
医生 1: 开始治疗患者 7 (已治疗: 8/30)
患者 13: 挂号成功, 候诊中 (当前候诊人数: 6)
医生 0: 开始治疗患者 8 (已治疗: 9/30)
患者 14: 挂号成功, 候诊中 (当前候诊人数: 6)
患者 15: 挂号成功, 候诊中 (当前候诊人数: 7)
医生 2: 完成治疗患者 6
患者 16: 挂号成功, 候诊中 (当前候诊人数: 8)
医生 1: 完成治疗患者 7
患者 17: 挂号成功, 候诊中 (当前候诊人数: 9)
医生 2: 开始治疗患者 9 (已治疗: 10/30)
医生 0: 完成治疗患者 8
患者 18: 挂号成功, 候诊中 (当前候诊人数: 9)
医生 1: 开始治疗患者 10 (已治疗: 11/30)
患者 19: 挂号成功, 候诊中 (当前候诊人数: 9)
医生 0: 开始治疗患者 11 (已治疗: 12/30)
患者 20: 挂号成功, 候诊中 (当前候诊人数: 9)
患者 21: 挂号成功, 候诊中 (当前候诊人数: 10)
医生 2: 完成治疗患者 9
患者 22: 挂号成功, 候诊中 (当前候诊人数: 11)
医生 1: 完成治疗患者 10

患者 23: 挂号成功，候诊中（当前候诊人数：12）
医生 2: 开始治疗患者 12（已治疗：13/30）

医生 0: 完成治疗患者 11
患者 24: 挂号成功, 候诊中 (当前候诊人数: 12)
医生 1: 开始治疗患者 13 (已治疗: 14/30)
患者 25: 挂号成功, 候诊中 (当前候诊人数: 12)
医生 0: 开始治疗患者 14 (已治疗: 15/30)
患者 26: 挂号成功, 候诊中 (当前候诊人数: 12)
患者 27: 挂号成功, 候诊中 (当前候诊人数: 13)
医生 2: 完成治疗患者 12
患者 28: 挂号成功, 候诊中 (当前候诊人数: 14)
医生 1: 完成治疗患者 13
患者 29: 挂号成功, 候诊中 (当前候诊人数: 15)
医生 2: 开始治疗患者 15 (已治疗: 16/30)
医生 0: 完成治疗患者 14
患者 30: 号已满, 离开医院
医生 1: 开始治疗患者 16 (已治疗: 17/30)
患者 31: 号已满, 离开医院
医生 0: 开始治疗患者 17 (已治疗: 18/30)
患者 32: 号已满, 离开医院
患者 33: 号已满, 离开医院
医生 2: 完成治疗患者 15
患者 34: 号已满, 离开医院
医生 1: 完成治疗患者 16
医生 2: 开始治疗患者 18 (已治疗: 19/30)
医生 0: 完成治疗患者 17
医生 1: 开始治疗患者 19 (已治疗: 20/30)
医生 0: 开始治疗患者 20 (已治疗: 21/30)
医生 2: 完成治疗患者 18
医生 1: 完成治疗患者 19
医生 2: 开始治疗患者 21 (已治疗: 22/30)
医生 0: 完成治疗患者 20
医生 1: 开始治疗患者 22 (已治疗: 23/30)
医生 0: 开始治疗患者 23 (已治疗: 24/30)
医生 2: 完成治疗患者 21
医生 1: 完成治疗患者 22
医生 2: 开始治疗患者 24 (已治疗: 25/30)
医生 0: 完成治疗患者 23
医生 1: 开始治疗患者 25 (已治疗: 26/30)
医生 0: 开始治疗患者 26 (已治疗: 27/30)
医生 2: 完成治疗患者 24

```
医生 1: 完成治疗患者 25
医生 2: 开始治疗患者 27 (已治疗: 28/30)
医生 0: 完成治疗患者 26
医生 1: 开始治疗患者 28 (已治疗: 29/30)
医生 0: 开始治疗患者 29 (已治疗: 30/30)
医生 2: 完成治疗患者 27
医生 1: 完成治疗患者 28
医生 0: 完成治疗患者 29
```

结果分析:

- 患者挂号、医生治疗的同步逻辑正确
- 互斥锁保护共享状态变量
- 条件变量实现高效等待

6.6 性能分析结果

分析结果:

实验6.1 - 检测CPU信息:

=== CPU信息检测 ===

物理CPU数量: 1

每个CPU的核数: 10

逻辑CPU数量: 16

CPU型号: model name : 12th Gen Intel(R) Core(TM) i7-12650H

实验6.2 - 基础并行对比:

开始性能测试...

正在测试: 单线程串行

第1次运行时间: 101318 微秒

第2次运行时间: 97288 微秒

第3次运行时间: 91137 微秒

第4次运行时间: 93303 微秒

第5次运行时间: 91991 微秒

=== 单线程串行 测试结果 ===

最佳运行时间: 91137 微秒

平均运行时间: 95007 微秒

K最佳运行时间: 95007 微秒

=====

正在测试: 两线程并行

第1次运行时间: 48406 微秒

第2次运行时间: 50430 微秒

第3次运行时间: 51716 微秒

第4次运行时间: 45717 微秒

第5次运行时间: 59011 微秒

=== 两线程并行 测试结果 ===

最佳运行时间: 45717 微秒

平均运行时间: 51056 微秒

```
K最佳运行时间：51056 微秒
=====

正在测试：三线程加锁
    第1次运行时间：1978588 微秒
    第2次运行时间：1994808 微秒
    第3次运行时间：2026595 微秒
    第4次运行时间：2429379 微秒
    第5次运行时间：2059759 微秒
=== 三线程加锁 测试结果 ===
最佳运行时间：1978588 微秒
平均运行时间：2097825 微秒
K最佳运行时间：2097825 微秒
=====
```

实验6.3 - 加锁vs不加锁：
开始性能测试...

```
正在测试：三线程加锁
    第1次运行时间：2131760 微秒
    第2次运行时间：2043256 微秒
    第3次运行时间：2108406 微秒
    第4次运行时间：2086398 微秒
    第5次运行时间：2146443 微秒
=== 三线程加锁 测试结果 ===
最佳运行时间：2043256 微秒
平均运行时间：2103252 微秒
K最佳运行时间：2103252 微秒
=====
```

```
正在测试：三线程不加锁
    第1次运行时间：231587 微秒
    第2次运行时间：246822 微秒
    第3次运行时间：257054 微秒
    第4次运行时间：262360 微秒
    第5次运行时间：267044 微秒
=== 三线程不加锁 测试结果 ===
最佳运行时间：231587 微秒
平均运行时间：252973 微秒
K最佳运行时间：252973 微秒
=====
```

实验6.4 - Cache优化：
=====

Cache优化性能测试

=====

测试配置：

- 循环次数：100000000
- 测试迭代：5次
- Cache行填充：128字节


```
=== 单线程基准测试 ===
开始单线程基准测试...
单线程结果: a=4999999950000000, b=4999999950000000
第1次运行时间: 109725 微秒 (0.110秒)
开始单线程基准测试...
单线程结果: a=4999999950000000, b=4999999950000000
第2次运行时间: 106998 微秒 (0.107秒)
开始单线程基准测试...
单线程结果: a=4999999950000000, b=4999999950000000
第3次运行时间: 106937 微秒 (0.107秒)
开始单线程基准测试... -
单线程结果: a=4999999950000000, b=4999999950000000
第4次运行时间: 101378 微秒 (0.101秒)
开始单线程基准测试...
单线程结果: a=4999999950000000, b=4999999950000000
第5次运行时间: 109068 微秒 (0.109秒)
```

```
=== 未优化版本测试 (可能存在Cache伪共享) ===
开始未优化版本测试...
未优化版本结果: a=4999999950000000, b=4999999950000000
第1次运行时间: 274597 微秒 (0.275秒)
开始未优化版本测试...
未优化版本结果: a=4999999950000000, b=4999999950000000
第2次运行时间: 258997 微秒 (0.259秒)
开始未优化版本测试...
未优化版本结果: a=4999999950000000, b=4999999950000000
第3次运行时间: 256991 微秒 (0.257秒)
开始未优化版本测试...
未优化版本结果: a=4999999950000000, b=4999999950000000
第4次运行时间: 249259 微秒 (0.249秒)
开始未优化版本测试...
未优化版本结果: a=4999999950000000, b=4999999950000000
第5次运行时间: 261498 微秒 (0.261秒)
```

```
=== 优化版本测试 (Cache行对齐) ===
开始优化版本测试...
优化版本结果: a=4999999950000000, b=4999999950000000
第1次运行时间: 61359 微秒 (0.061秒)
开始优化版本测试...
优化版本结果: a=4999999950000000, b=4999999950000000
第2次运行时间: 60946 微秒 (0.061秒)
开始优化版本测试...
优化版本结果: a=4999999950000000, b=4999999950000000
第3次运行时间: 54806 微秒 (0.055秒)
开始优化版本测试...
优化版本结果: a=4999999950000000, b=4999999950000000
第4次运行时间: 61201 微秒 (0.061秒)
开始优化版本测试... -
优化版本结果: a=4999999950000000, b=4999999950000000
第5次运行时间: 63205 微秒 (0.063秒)
```

=====

测试结果分析

=====

平均运行时间:

单线程基准: 106821 微秒
未优化版本: 260268 微秒
优化版本: 60303 微秒

K最佳运行时间:

单线程基准: 106821 微秒
未优化版本: 260268 微秒
优化版本: 60303 微秒

性能对比:

Cache优化带来性能提升: +76.83%

相对于单线程:

未优化版本并行加速比: 0.41x
优化版本并行加速比: 1.77x

实验6.5 - CPU 亲和力对并行程序影响:

=== 系统CPU信息 ===

CPU(s) scaling MHz: 40%

=== CPU架构信息 ===

CPU(s) scaling MHz: 31%

CPU 最大 MHz: 4700.0000

CPU 最小 MHz: 400.0000

=====

开始CPU亲和性性能测试...

每个测试将运行 5 次, 取最佳5次的平均值

-

=== 开始测试: 单线程基准 ===

单线程测试完成: a=1249999975000000, b=1249999975000000

第1次运行时间: 55081 微秒

单线程测试完成: a=1249999975000000, b=1249999975000000

第2次运行时间: 53294 微秒

单线程测试完成: a=1249999975000000, b=1249999975000000

第3次运行时间: 54127 微秒

单线程测试完成: a=1249999975000000, b=1249999975000000

第4次运行时间: 55142 微秒

单线程测试完成: a=1249999975000000, b=1249999975000000

第5次运行时间: 56047 微秒

=== 单线程基准 测试结果 ===

最佳运行时间: 53294 微秒

平均运行时间: 54738 微秒

K最佳运行时间: 54738 微秒

=====

=== 开始测试: 无CPU亲和性(两线程) ===

无亲和性测试完成: a=1249999975000000, b=1249999975000000

第1次运行时间: 34176 微秒

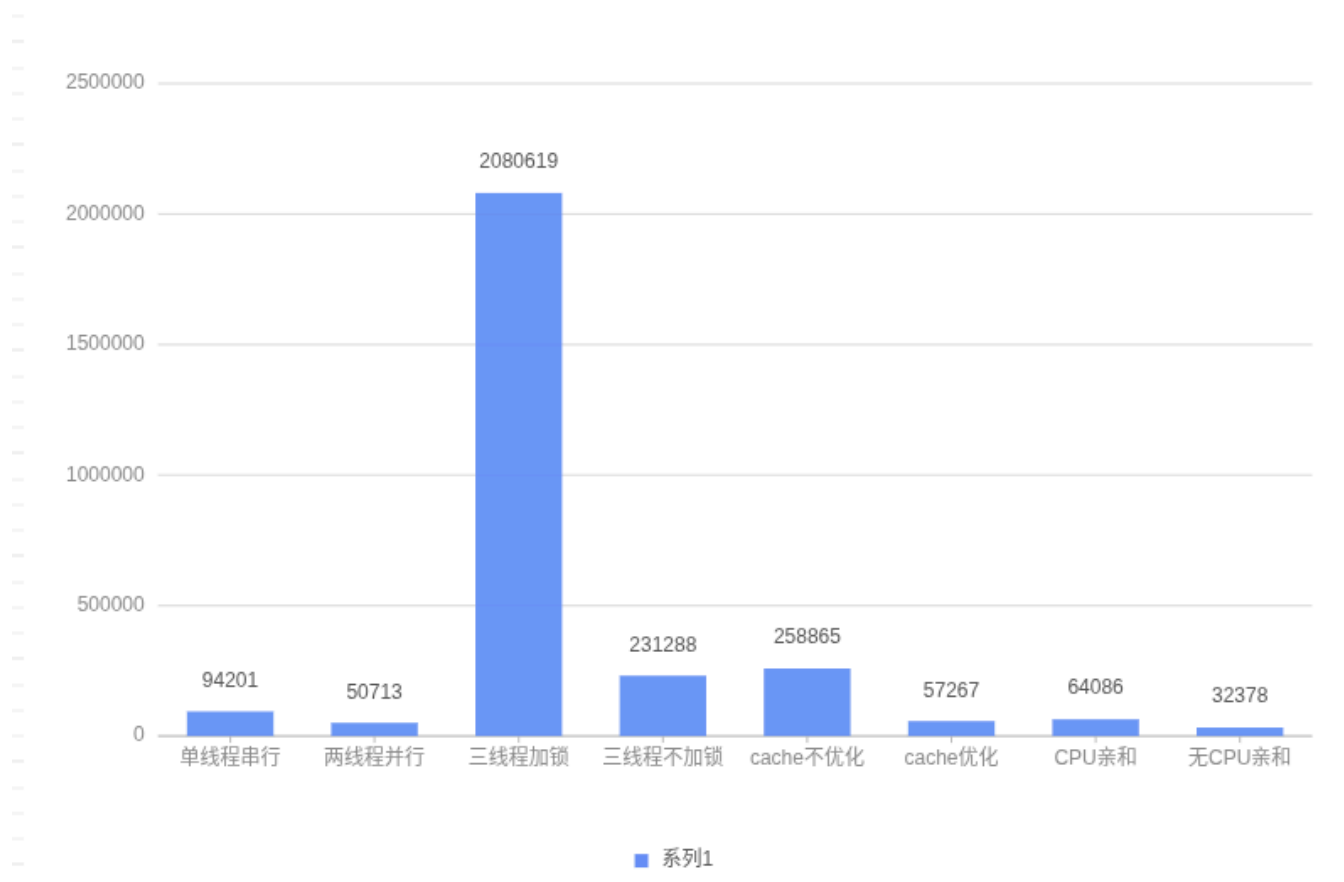
无亲和性测试完成: a=1249999975000000, b=1249999975000000

```
第2次运行时间：30085 微秒
无亲和性测试完成：a=1249999975000000, b=1249999975000000
第3次运行时间：30188 微秒
无亲和性测试完成：a=1249999975000000, b=1249999975000000
第4次运行时间：35871 微秒
无亲和性测试完成：a=1249999975000000, b=1249999975000000
第5次运行时间：36386 微秒
=== 无CPU亲和性(两线程) 测试结果 ===
最佳运行时间：30085 微秒
平均运行时间：33341 微秒
K最佳运行时间：33341 微秒
=====

=== 开始测试：有CPU亲和性(两线程) ===
有亲和性测试完成：a=1249999975000000, b=1249999975000000
第1次运行时间：64669 微秒
有亲和性测试完成：a=1249999975000000, b=1249999975000000
第2次运行时间：67569 微秒
有亲和性测试完成：a=1249999975000000, b=1249999975000000
第3次运行时间：64119 微秒
有亲和性测试完成：a=1249999975000000, b=1249999975000000
第4次运行时间：65401 微秒
有亲和性测试完成：a=1249999975000000, b=1249999975000000
第5次运行时间：67746 微秒
=== 有CPU亲和性(两线程) 测试结果 ===
最佳运行时间：64119 微秒
平均运行时间：65900 微秒
K最佳运行时间：65900 微秒
=====
```

性能对比数据（示例）：

测试方案	平均运行时间(μs)	相对单线程加速比
单线程串行	94201	1.0x
两线程并行	50713	1.86x
三线程加锁	2080619	0.05x
三线程不加锁	231288	0.41x
Cache不优化版	258865	0.36x
Cache优化版	57267	1.65x
CPU亲和性(添加cache优化)	64086	1.47x
未设置CPU亲和性(添加cache优化)	32378	2.91x



性能分析结论：

1. **并行化效果**：两线程并行相比单线程有明显加速
2. **锁开销**：三线程加锁版本因锁竞争导致性能下降
3. **Cache优化**：通过数据对齐减少伪共享，提升明显
4. **CPU亲和性**：绑定线程到特定CPU核心，使得系统降低自主调度权重，运行时间增加

7. 实验总结

7.1 主要收获

1. **深入理解进程线程概念**：通过实践加深了对进程和线程区别的理解
2. **掌握同步互斥机制**：熟练使用互斥锁、条件变量等同步原语
3. **熟悉进程间通信**：掌握了管道、共享内存等通信机制
4. **性能分析能力**：学会了多线程程序的性能调优方法

7.2 遇到问题

1. **性能优化挑战**：Cache伪共享问题通过数据对齐优化
2. **CPU亲和性结果反直觉**：按照理论分析，增加CPU亲和性后会减少程序运行时的调度开销，从而运行时间变少。但是根据实验结果，增加CPU亲和性后时间反而增加了一倍。经过分析，可能是因为我为子线程1增加CPU0的亲和性，为子线程2增加CPU1的亲和性，使得两个线程在实际运行时取消自动调度，转而仅使用亲和的CPU进行运行，而本来同一个线程是可以同时调度两个CPU进行执行的。另外，CPU0和CPU1在逻辑上是连续的CPU，导致了增加1倍的运行时间；如果将CPU亲和改为1号和5号，此时CPU不连续，则添加亲和性和不添加运行时间相近。

3. 对Linux下函数库的不熟悉，以及缺乏系统编程的经验，使得刚开始实验时完全无法下手。借助实验指导书，查阅相关博客，以及借助大模型进行学习后渐渐上手，对系统变成的了解增加。
4. 信号量设置不正确导致结果不对，导致程序死锁。通过重温三个经典问题的设计过程，熟悉信号量的机制，对代码进行debug。

附录：主要程序代码

实验1.2 进程创建与管理 lab1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

int main() {
    pid_t pid1, pid2;
    int status;

    printf("父进程PID: %d\n", getpid());

    if ((pid1 = fork()) == 0) {
        printf("子进程1 PID: %d, 父进程PID: %d\n", getpid(), getppid());

        char *args[] = {"ls", "-l", NULL};
        printf("子进程1调用exec执行ls -l命令:\n");
        execvp("ls", args);

        perror("exec失败");
        exit(1);
    } else if (pid1 > 0) {

        if ((pid2 = fork()) == 0) {
            printf("子进程2 PID: %d\n", getpid());

            for (int i = 0; i < 5; i++) {
                printf("子进程2正在工作...%d\n", i);
                sleep(2);
            }

            printf("子进程2正常退出\n");
            exit(10);
        } else if (pid2 > 0) {
            printf("父进程创建了两个子进程: %d 和 %d\n", pid1, pid2);

            sleep(8);
            printf("\n父进程尝试终止子进程2 (PID: %d)...\n", pid2);

            if (kill(pid2, SIGTERM) == 0) {
                printf("已向子进程2发送SIGTERM信号\n");
            }
        }
    }
}
```

```

    } else {
        perror("kill失败");
    }

    pid_t finished_pid = wait(&status);
    if (finished_pid == pid1) {
        printf("子进程1 (PID: %d) 已结束\n", pid1);
    } else {
        if (WIFEXITED(status)) {
            printf("子进程2 (PID: %d) 正常退出, 退出状态: %d\n",
                finished_pid, WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("子进程2 (PID: %d) 被信号终止, 信号编号: %d\n",
                finished_pid, WTERMSIG(status));
        }
    }
}

finished_pid = wait(&status);
if (finished_pid == pid1) {
    printf("子进程1 (PID: %d) 已结束\n", pid1);
} else {
    if (WIFEXITED(status)) {
        printf("子进程2 (PID: %d) 正常退出, 退出状态: %d\n",
            finished_pid, WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("子进程2 (PID: %d) 被信号终止, 信号编号: %d\n",
            finished_pid, WTERMSIG(status));
    }
}

printf("所有子进程已结束, 父进程退出\n");
} else {
    perror("第二个fork失败");
    exit(1);
}
} else {
    perror("第一个fork失败");
    exit(1);
}

return 0;
}

```

实验1.3 进程管理命令 lab1-3.sh

```

#!/bin/bash

echo "=== 进程管理命令快速演示 ==="
echo ""

echo "1. ps命令演示:"
echo "当前进程:"

```

```
ps
echo ""

echo "2. top命令演示（快速显示）："
top -bn1 | head -10
echo ""

echo "3. pstree命令演示："
pstree | head -5
echo ""

echo "4. vmstat命令演示："
vmstat 1 2
echo ""

echo "5. 创建测试进程用于演示："
sleep 60 &
TEST_PID=$!
echo "创建sleep进程，PID: $TEST_PID"
echo ""

echo "6. jobs命令演示："
jobs
echo ""

echo "7. kill命令演示："
echo "终止测试进程..."
kill $TEST_PID
echo ""

echo "8. strace命令演示："
strace -c echo "test" 2>&1 | head -5
echo ""

echo "9. ltrace命令演示："
if command -v ltrace &> /dev/null; then
    ltrace echo "test" 2>&1 | head -5
else
    echo "ltrace未安装"
fi
echo ""

echo "10. sleep命令演示："
echo "等待2秒..."
sleep 2
echo "等待结束"
echo ""

echo "=== 演示结束 ==="
```

实验2.2 线程创建与管理 lab2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

void* thread_func1(void* arg) {
    int thread_id = *(int*)arg;
    printf("线程%d (ID: %lu) 启动 - 使用return退出\n",
        thread_id, pthread_self());

    for (int i = 0; i < 3; i++) {
        printf("线程%d工作: %d\n", thread_id, i);
        usleep(300000);
    }

    printf("线程%d通过return退出\n", thread_id);
    return (void*)(long)(thread_id * 100);
}

void* thread_func2(void* arg) {
    int thread_id = *(int*)arg;
    printf("线程%d (ID: %lu) 启动 - 使用pthread_exit退出\n",
        thread_id, pthread_self());

    for (int i = 0; i < 3; i++) {
        printf("线程%d工作: %d\n", thread_id, i);
        usleep(200000);
    }

    printf("线程%d通过pthread_exit退出\n", thread_id);
    pthread_exit((void*)(long)(thread_id * 200));
}

void* thread_func3(void* arg) {
    int thread_id = *(int*)arg;
    printf("线程%d (ID: %lu) 启动 - 将被主线程取消\n",
        thread_id, pthread_self());

    pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);

    for (int i = 0; i < 3; i++) {
        printf("线程%d工作: %d (线程ID: %lu)\n", thread_id, i, pthread_self());
        usleep(400000);

        pthread_testcancel();
    }

    printf("线程%d正常结束 (如果被取消, 不会看到这条) \n", thread_id);
    return (void*)(long)(thread_id * 300);
}
```



```

}

int main() {
    pthread_t thread1, thread2, thread3;
    int id1 = 1, id2 = 2, id3 = 3;
    void *ret1, *ret2, *ret3;

    printf("主线程ID: %lu\n", pthread_self());
    printf("=== 开始创建线程 ===\n");

    pthread_create(&thread1, NULL, thread_func1, &id1);

    pthread_create(&thread2, NULL, thread_func2, &id2);

    pthread_create(&thread3, NULL, thread_func3, &id3);

    printf("=== 所有线程创建完成 ===\n");
    printf("线程1 ID: %lu\n", thread1);
    printf("线程2 ID: %lu\n", thread2);
    printf("线程3 ID: %lu\n", thread3);

    printf("\n主线程主动取消线程3...\n");
    pthread_cancel(thread3);

    pthread_join(thread1, &ret1);
    printf("线程1返回值: %ld\n", (long)ret1);

    pthread_join(thread2, &ret2);
    printf("线程2返回值: %ld\n", (long)ret2);

    pthread_join(thread3, &ret3);
    if (ret3 == PTHREAD_CANCELED) {
        printf("线程3: 被主线程取消终止 (PTHREAD_CANCELED)\n");
    } else {
        printf("线程3返回值: %ld\n", (long)ret3);
    }

    printf("\n=== 所有线程执行完成 ===\n");
    printf("主线程结束\n");

    return 0;
}

```

实验3 进程通信 lab3.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>

```

```

#include <sys/wait.h>

#define FIFO_NAME "/tmp/myfifo"
#define BUFFER_SIZE 256

void writer_process() {
    int fd;
    char buffer[BUFFER_SIZE];

    printf("写者进程启动 (PID: %d)\n", getpid());

    fd = open(FIFO_NAME, O_WRONLY);
    if (fd == -1) {
        perror("写者打开管道失败");
        exit(1);
    }

    char *messages[] = {
        "第一条消息: Hello from writer!",
        "第二条消息: 命名管道通信测试",
        "第三条消息: 进程间通信成功! ",
        "END"
    };

    for (int i = 0; i < 4; i++) {
        printf("写者发送: %s\n", messages[i]);
        write(fd, messages[i], strlen(messages[i]) + 1);
        sleep(1);
    }

    close(fd);
    printf("写者进程结束\n");
}

void reader_process() {
    int fd;
    char buffer[BUFFER_SIZE];
    int bytes_read;

    printf("读者进程启动 (PID: %d)\n", getpid());

    fd = open(FIFO_NAME, O_RDONLY);
    if (fd == -1) {
        perror("读者打开管道失败");
        exit(1);
    }

    while (1) {
        bytes_read = read(fd, buffer, BUFFER_SIZE);
        if (bytes_read > 0) {
            printf("读者接收: %s\n", buffer);

            if (strcmp(buffer, "END") == 0) {

```

```

        break;
    }
}

close(fd);
printf("读者进程结束\n");
}

void named_pipe(){
    pid_t pid1, pid2;

    printf("=== 命名管道通信示例 ===\n");

    if (access(FIFO_NAME, F_OK) == -1) {
        if (mkfifo(FIFO_NAME, 0666) == -1) {
            perror("命名管道创建失败");
            exit(1);
        }
        printf("命名管道创建成功: %s\n", FIFO_NAME);
    } else {
        printf("命名管道已存在: %s\n", FIFO_NAME);
    }

    pid1 = fork();
    if (pid1 == 0) {
        writer_process();
        exit(0);
    }

    pid2 = fork();
    if (pid2 == 0) {
        reader_process();
        exit(0);
    }

    wait(NULL);
    wait(NULL);

    unlink(FIFO_NAME);
    printf("命名管道已删除\n");
    printf("主进程结束\n");
}

void unnamed_pipe(){
    int pipe_fd[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];
    int bytes_read;

    printf("=== 无名管道通信示例 ===\n");
    printf("父进程PID: %d\n", getpid());

```

```

if (pipe(pipe_fd) == -1) {
    perror("管道创建失败");
    exit(1);
}

printf("管道创建成功：读端=%d，写端=%d\n", pipe_fd[0], pipe_fd[1]);

pid = fork();

if (pid == -1) {
    perror("fork失败");
    exit(1);
}

if (pid == 0) {
    printf("子进程启动 (PID: %d)\n", getpid());

    close(pipe_fd[0]);

    lockf(pipe_fd[1], F_LOCK, 0);

    char message[] = "Hello from child process! 这是子进程通过管道发送的消息。";
    write(pipe_fd[1], message, strlen(message) + 1);
    printf("子进程写入数据：%s\n", message);

    lockf(pipe_fd[1], F_ULOCK, 0);

    sleep(1);
    char message2[] = "这是子进程的第二条消息! ";
    write(pipe_fd[1], message2, strlen(message2) + 1);
    printf("子进程写入第二条数据：%s\n", message2);

    close(pipe_fd[1]);
    printf("子进程结束\n");
    exit(0);
} else {
    printf("父进程等待子进程数据...\n");

    close(pipe_fd[1]);

    while ((bytes_read = read(pipe_fd[0], buffer, BUFFER_SIZE)) > 0) {
        printf("父进程接收到数据：%s\n", buffer);
        printf("接收字节数：%d\n", bytes_read);
    }

    wait(NULL);

    close(pipe_fd[0]);
    printf("父进程结束\n");
}
}

```

```
int main() {
    named_pipe();
    sleep(1);
    unnamed_pipe();
    return 0;
}
```

实验4 线程通信 lab4.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>

#define NUM_THREADS 3

typedef struct {
    int count;
    char message[100];
    pthread_mutex_t mutex;
} shared_data_t;

shared_data_t shared_data;

void* writer_thread(void* arg) {
    int thread_id = *(int*)arg;

    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&shared_data.mutex);

        shared_data.count++;
        sprintf(shared_data.message, "消息来自线程%d, 计数: %d", thread_id, shared_data.count);
        printf("线程%d写入: %s\n", thread_id, shared_data.message);

        pthread_mutex_unlock(&shared_data.mutex);
        usleep(200000);
    }

    return NULL;
}

void* reader_thread(void* arg) {
    int thread_id = *(int*)arg;

    for (int i = 0; i < 4; i++) {
        pthread_mutex_lock(&shared_data.mutex);

        printf("线程%d读取: 计数=%d, 消息=%s\n",
            thread_id, shared_data.count, shared_data.message);

        pthread_mutex_unlock(&shared_data.mutex);
    }
}
```

```

        usleep(300000);
    }

    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS] = {1, 2, 3};

    shared_data.count = 0;
    strcpy(shared_data.message, "初始消息");
    pthread_mutex_init(&shared_data.mutex, NULL);

    printf("主线程启动\n");
    printf("初始共享数据: 计数=%d, 消息=%s\n", shared_data.count, shared_data.message);

    if (pthread_create(&threads[0], NULL, writer_thread, &thread_ids[0]) != 0) {
        perror("创建写入线程失败");
        return 1;
    }

    if (pthread_create(&threads[1], NULL, reader_thread, &thread_ids[1]) != 0) {
        perror("创建读取线程1失败");
        return 1;
    }

    if (pthread_create(&threads[2], NULL, reader_thread, &thread_ids[2]) != 0) {
        perror("创建读取线程2失败");
        return 1;
    }

    // 主进程进行通信
    pthread_mutex_lock(&shared_data.mutex);
    shared_data.count*=2;
    sprintf(shared_data.message, "消息来自主线程%d, 计数: %d", getpid(), shared_data.count);
    printf("主线程%d写入: %s\n", getpid(), shared_data.message);
    pthread_mutex_unlock(&shared_data.mutex);
    usleep(300000);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("最终共享数据: 计数=%d, 消息=%s\n", shared_data.count, shared_data.message);

    pthread_mutex_destroy(&shared_data.mutex);
    printf("主线程结束\n");

    return 0;
}

```

实验5 同步与互斥 lab5.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define N 30
#define M 3

pthread_mutex_t mutex_p = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex_d = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t patient_cond = PTHREAD_COND_INITIALIZER;

typedef struct {
    int id;
    int state;
}patient;

patient Pstate[N];
int wait_count = 0;
int Dstate[M] = {0};
int treated_patients = 0;
int patient_id_counter = 0;

void* patient_thread(void* arg) {
    int id = *(int*)arg;

    pthread_mutex_lock(&mutex_p);

    if (patient_id_counter >= N) {
        printf("患者 %d: 号已满, 离开医院\n", id);
        pthread_mutex_unlock(&mutex_p);
        return NULL;
    }

    patient tem;
    tem.id=id;
    tem.state=1;
    Pstate[patient_id_counter++] = tem;
    wait_count++;
    printf("患者 %d: 挂号成功, 候诊中 (当前候诊人数: %d) \n", id, wait_count);

    pthread_cond_signal(&patient_cond);
    pthread_mutex_unlock(&mutex_p);

    return NULL;
}

void* doctor_thread(void* arg) {
    int doctor_id = *(int*)arg;
```

```

while (1) {
    pthread_mutex_lock(&mutex_d);

    while (wait_count == 0 || Dstate[doctor_id] == 1) {
        if (treated_patients >= N) {
            pthread_mutex_unlock(&mutex_d);
            return NULL;
        }
        pthread_cond_wait(&patient_cond, &mutex_d);
    }

    if (treated_patients >= N) {
        pthread_mutex_unlock(&mutex_d);
        break;
    }

    int patient_id, curp;

    pthread_mutex_lock(&mutex_p);
    for(int i=0; i<N; i++){
        if(Pstate[i].state==1){
            curp=i;
            patient_id=Pstate[i].id;
            Pstate[i].state=2;
            break;
        }
    }
    pthread_mutex_unlock(&mutex_p);

    wait_count--;

    Dstate[doctor_id] = 1;
    treated_patients++;

    printf("医生 %d: 开始治疗患者 %d (已治疗: %d/%d) \n",
        doctor_id, patient_id, treated_patients, N);

    pthread_mutex_unlock(&mutex_d);

    usleep(200000);

    pthread_mutex_lock(&mutex_p);
    Pstate[curp].state=3;
    pthread_mutex_unlock(&mutex_p);

    pthread_mutex_lock(&mutex_d);
    Dstate[doctor_id] = 0;
    printf("医生 %d: 完成治疗患者 %d\n", doctor_id, patient_id);

    pthread_cond_signal(&patient_cond);
    pthread_mutex_unlock(&mutex_d);

    usleep(100000);
}

```



```

    }

    return NULL;
}

int main() {
    pthread_t doctors[M];
    pthread_t patients[N + 5];

    int doctor_ids[M];
    int patient_ids[N + 5];

    for (int i = 0; i < M; i++) {
        doctor_ids[i] = i;
        pthread_create(&doctors[i], NULL, doctor_thread, &doctor_ids[i]);
    }

    for (int i = 0; i < N + 5; i++) {
        patient_ids[i] = i;
        pthread_create(&patients[i], NULL, patient_thread, &patient_ids[i]);
        usleep(50000);
    }

    for (int i = 0; i < N + 5; i++) {
        pthread_join(patients[i], NULL);
    }

    pthread_cond_broadcast(&patient_cond);

    for (int i = 0; i < M; i++) {
        pthread_join(doctors[i], NULL);
    }

    printf("\n=== 今日门诊结束 ===\n");
    printf("总治疗患者: %d/%d\n", treated_patients, N);
    printf("剩余候诊患者: %d\n", wait_count);

    pthread_mutex_destroy(&mutex_p);
    pthread_mutex_destroy(&mutex_d);
    pthread_cond_destroy(&patient_cond);

    return 0;
}

```

实验6-1 观察CPU lab6-1.c

```

#include <stdio.h>
#include <unistd.h>

void get_cpu_info() {
    printf("=== CPU信息检测 ===\n");
}

```

```

FILE *fp = popen("grep 'physical id' /proc/cpuinfo | sort | uniq | wc -l", "r");
char physical_cpu[10];
fgets(physical_cpu, sizeof(physical_cpu), fp);
pclose(fp);
printf("物理CPU数量: %s", physical_cpu);

fp = popen("grep 'cpu cores' /proc/cpuinfo | uniq | awk -F: '{print $2}'", "r");
char cpu_cores[10];
fgets(cpu_cores, sizeof(cpu_cores), fp);
pclose(fp);
printf("每个CPU的核数: %s", cpu_cores);

fp = popen("cat /proc/cpuinfo | grep 'processor' | wc -l", "r");
char logical_cpu[10];
fgets(logical_cpu, sizeof(logical_cpu), fp);
pclose(fp);
printf("逻辑CPU数量: %s", logical_cpu);

fp = popen("grep 'model name' /proc/cpuinfo | uniq", "r");
char model_name[100];
fgets(model_name, sizeof(model_name), fp);
pclose(fp);
printf("CPU型号: %s", model_name);
}

int main() {
    get_cpu_info();
    return 0;
}

```

实验6-2 单线程/进程串行 vs 2 线程并行 vs 3 线程加锁并行程序对比 lab6-2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>

#define ORANGE_MAX_VALUE 1000000
#define APPLE_MAX_VALUE 100000000
#define MSECOND 1000000

struct apple {
    unsigned long long a;
    unsigned long long b;
    pthread_rwlock_t rwlock;
};

struct orange {
    int a[ORANGE_MAX_VALUE];
    int b[ORANGE_MAX_VALUE];
};

```

```

long long get_current_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (long long)tv.tv_sec * 1000000 + tv.tv_usec;
}

void* single_thread_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    struct orange* test1 = (struct orange*)malloc(sizeof(struct orange));

    test->a = 0;
    test->b = 0;

    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
        test->b += sum;
    }

    unsigned long long sum = 0;
    for (int i = 0; i < ORANGE_MAX_VALUE; i++) {
        sum += test1->a[i] + test1->b[i];
    }

    free(test);
    free(test1);
    return NULL;
}

void* calc_apple(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }
    return NULL;
}

void* calc_apple_b_lock(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        pthread_rwlock_wrlock(&test->rwlock);
        test->b += sum;
        pthread_rwlock_unlock(&test->rwlock);
    }
    return NULL;
}

void* calc_orange(void* arg) {
    struct orange* test1 = (struct orange*)arg;
    unsigned long long sum = 0;
    for (int i = 0; i < ORANGE_MAX_VALUE; i++) {
        sum += test1->a[i] + test1->b[i];
    }
}

```

```

        return NULL;
    }

void* two_thread_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    struct orange* test1 = (struct orange*)malloc(sizeof(struct orange));

    test->a = 0;
    test->b = 0;

    pthread_t thread_apple, thread_orange;

    pthread_create(&thread_apple, NULL, calc_apple, test);
    pthread_create(&thread_orange, NULL, calc_orange, test1);

    pthread_join(thread_apple, NULL);
    pthread_join(thread_orange, NULL);

    free(test);
    free(test1);
    return NULL;
}

void* three_thread_lock_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    struct orange* test1 = (struct orange*)malloc(sizeof(struct orange));

    test->a = 0;
    test->b = 0;
    pthread_rwlock_init(&test->rwlock, NULL);

    pthread_t thread_apple_a, thread_apple_b, thread_orange;

    pthread_create(&thread_apple_a, NULL, calc_apple, test);
    pthread_create(&thread_apple_b, NULL, calc_apple_b_lock, test);
    pthread_create(&thread_orange, NULL, calc_orange, test1);

    pthread_join(thread_apple_a, NULL);
    pthread_join(thread_apple_b, NULL);
    pthread_join(thread_orange, NULL);

    pthread_rwlock_destroy(&test->rwlock);
    free(test);
    free(test1);
    return NULL;
}

int compare_long_long(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

```

```

}

long long calculate_k_best(long long* times, int count, int k) {
    qsort(times, count, sizeof(long long), compare_long_long);
    long long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += times[i];
    }
    return sum / k;
}

void run_test(const char* test_name, void* (*test_func)(void*), int iterations) {
    printf("正在测试: %s\n", test_name);

    long long* times = (long long*)malloc(iterations * sizeof(long long));

    for (int i = 0; i < iterations; i++) {
        long long start_time = get_current_time();
        test_func(NULL);
        long long end_time = get_current_time();
        long long duration = end_time - start_time;

        times[i] = duration;

        printf("  第%d次运行时间: %lld 微秒\n", i+1, duration);
    }

    long long k_best_time = calculate_k_best(times, iterations, iterations > 5 ? 5 :
iterations);

    printf("=== %s 测试结果 ===\n", test_name);
    printf("最佳运行时间: %lld 微秒\n", times[0]);
    printf("平均运行时间: %lld 微秒\n", calculate_k_best(times, iterations, iterations));
    printf("K最佳运行时间: %lld 微秒\n", k_best_time);
    printf("=====\n\n");

    free(times);
}

int main() {
    int iterations = 5;

    printf("开始性能测试...\n\n");

    run_test("单线程串行", single_thread_test, iterations);

    run_test("两线程并行", two_thread_test, iterations);

    run_test("三线程加锁", three_thread_lock_test, iterations);

    return 0;
}

```

实验6-3 线程加锁 vs 3 线程不加锁 lab6-3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>

#define ORANGE_MAX_VALUE 1000000
#define APPLE_MAX_VALUE 100000000
#define MSECOND 1000000

struct apple {
    unsigned long long a;
    unsigned long long b;
    pthread_rwlock_t rwlock;
};

struct orange {
    int a[ORANGE_MAX_VALUE];
    int b[ORANGE_MAX_VALUE];
};

long long get_current_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (long long)tv.tv_sec * 1000000 + tv.tv_usec;
}

void* calc_apple(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }
    return NULL;
}

void* calc_apple_b_lock(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        pthread_rwlock_wrlock(&test->rwlock);
        test->b += sum;
        pthread_rwlock_unlock(&test->rwlock);
    }
    return NULL;
}

void* calc_apple_b_no_lock(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->b += sum;
    }
}
```

```

    return NULL;
}

void* calc_orange(void* arg) {
    struct orange* test1 = (struct orange*)arg;
    unsigned long long sum = 0;
    for (int i = 0; i < ORANGE_MAX_VALUE; i++) {
        sum += test1->a[i] + test1->b[i];
    }
    return NULL;
}

void* three_thread_lock_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    struct orange* test1 = (struct orange*)malloc(sizeof(struct orange));

    test->a = 0;
    test->b = 0;
    pthread_rwlock_init(&test->rwlock, NULL);

    pthread_t thread_apple_a, thread_apple_b, thread_orange;

    pthread_create(&thread_apple_a, NULL, calc_apple, test);
    pthread_create(&thread_apple_b, NULL, calc_apple_b_lock, test);
    pthread_create(&thread_orange, NULL, calc_orange, test1);

    pthread_join(thread_apple_a, NULL);
    pthread_join(thread_apple_b, NULL);
    pthread_join(thread_orange, NULL);

    pthread_rwlock_destroy(&test->rwlock);
    free(test);
    free(test1);
    return NULL;
}

void* three_thread_no_lock_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    struct orange* test1 = (struct orange*)malloc(sizeof(struct orange));

    test->a = 0;
    test->b = 0;

    pthread_t thread_apple_a, thread_apple_b, thread_orange;

    pthread_create(&thread_apple_a, NULL, calc_apple, test);
    pthread_create(&thread_apple_b, NULL, calc_apple_b_no_lock, test);
    pthread_create(&thread_orange, NULL, calc_orange, test1);

    pthread_join(thread_apple_a, NULL);
    pthread_join(thread_apple_b, NULL);
    pthread_join(thread_orange, NULL);

```

```

    free(test);
    free(test1);
    return NULL;
}

int compare_long_long(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

long long calculate_k_best(long long* times, int count, int k) {
    qsort(times, count, sizeof(long long), compare_long_long);
    long long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += times[i];
    }
    return sum / k;
}

void run_test(const char* test_name, void* (*test_func)(void*), int iterations) {
    printf("正在测试: %s\n", test_name);

    long long* times = (long long*)malloc(iterations * sizeof(long long));

    for (int i = 0; i < iterations; i++) {
        long long start_time = get_current_time();
        test_func(NULL);
        long long end_time = get_current_time();
        long long duration = end_time - start_time;

        times[i] = duration;

        printf("  第%d次运行时间: %lld 微秒\n", i+1, duration);
    }

    long long k_best_time = calculate_k_best(times, iterations, iterations > 5 ? 5 :
iterations);

    printf("=== %s 测试结果 ===\n", test_name);
    printf("最佳运行时间: %lld 微秒\n", times[0]);
    printf("平均运行时间: %lld 微秒\n", calculate_k_best(times, iterations, iterations));
    printf("K最佳运行时间: %lld 微秒\n", k_best_time);
    printf("=====\n\n");

    free(times);
}

int main() {
    int iterations = 5;

```



```

printf("开始性能测试...\n\n");

run_test("三线程加锁", three_thread_lock_test, iterations);

run_test("三线程不加锁", three_thread_no_lock_test, iterations);

return 0;
}

```

实验6-4 针对 Cache 的优化 lab6-4.c

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>

#define APPLE_MAX_VALUE 100000000

struct apple_no_optimize {
    unsigned long long a;
    unsigned long long b;
    pthread_rwlock_t rwlock;
};

struct apple_optimized {
    unsigned long long a;
    char padding1[128];
    unsigned long long b;
    char padding2[128];
    pthread_rwlock_t rwlock;
};

long long get_current_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (long long)tv.tv_sec * 1000000 + tv.tv_usec;
}

void* calc_apple_a_no_opt(void* arg) {
    struct apple_no_optimize* test = (struct apple_no_optimize*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }
    return NULL;
}

void* calc_apple_b_no_opt(void* arg) {
    struct apple_no_optimize* test = (struct apple_no_optimize*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->b += sum;
    }
}

```

```

        return NULL;
    }

void* calc_apple_a_opt(void* arg) {
    struct apple_optimized* test = (struct apple_optimized*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }
    return NULL;
}

void* calc_apple_b_opt(void* arg) {
    struct apple_optimized* test = (struct apple_optimized*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->b += sum;
    }
    return NULL;
}

void* no_cache_optimize_test(void* arg) {
    struct apple_no_optimize* test = (struct apple_no_optimize*)malloc(sizeof(struct
apple_no_optimize));
    test->a = 0;
    test->b = 0;

    pthread_t thread1, thread2;

    printf("开始未优化版本测试...\n");
    pthread_create(&thread1, NULL, calc_apple_a_no_opt, test);
    pthread_create(&thread2, NULL, calc_apple_b_no_opt, test);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("未优化版本结果: a=%llu, b=%llu\n", test->a, test->b);

    free(test);
    return NULL;
}

void* cache_optimize_test(void* arg) {
    struct apple_optimized* test = (struct apple_optimized*)malloc(sizeof(struct
apple_optimized));
    test->a = 0;
    test->b = 0;

    pthread_t thread1, thread2;

    printf("开始优化版本测试...\n");
    pthread_create(&thread1, NULL, calc_apple_a_opt, test);
    pthread_create(&thread2, NULL, calc_apple_b_opt, test);

    pthread_join(thread1, NULL);

```

```

pthread_join(thread2, NULL);

printf("优化版本结果: a=%llu, b=%llu\n", test->a, test->b);

free(test);
return NULL;
}

void* single_thread_baseline(void* arg) {
    struct apple_no_optimize* test = (struct apple_no_optimize*)malloc(sizeof(struct
apple_no_optimize));
    test->a = 0;
    test->b = 0;

    printf("开始单线程基准测试...\n");

    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->b += sum;
    }

    printf("单线程结果: a=%llu, b=%llu\n", test->a, test->b);

    free(test);
    return NULL;
}

int compare_long_long(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

long long calculate_k_best(long long* times, int count, int k) {
    qsort(times, count, sizeof(long long), compare_long_long);
    long long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += times[i];
    }
    return sum / k;
}

void run_cache_test() {
    int iterations = 5;

    printf("=====\n");
    printf("          Cache优化性能测试\n");
    printf("=====\n\n");
}

```

```

printf("测试配置:\n");
printf("- 循环次数: %d\n", APPLE_MAX_VALUE);
printf("- 测试迭代: %d次\n", iterations);
printf("- Cache行填充: 128字节\n\n");

long long* single_times = (long long*)malloc(iterations * sizeof(long long));
long long* no_opt_times = (long long*)malloc(iterations * sizeof(long long));
long long* opt_times = (long long*)malloc(iterations * sizeof(long long));

printf("=== 单线程基准测试 ===\n");
for (int i = 0; i < iterations; i++) {
    long long start = get_current_time();
    single_thread_baseline(NULL);
    long long end = get_current_time();
    long long duration = end - start;
    single_times[i] = duration;
    printf("第%d次运行时间: %lld 微秒 (%.3f秒)\n", i+1, duration, duration/1000000.0);
    usleep(100000);
}

printf("\n=== 未优化版本测试 (可能存在Cache伪共享) ===\n");
for (int i = 0; i < iterations; i++) {
    long long start = get_current_time();
    no_cache_optimize_test(NULL);
    long long end = get_current_time();
    long long duration = end - start;
    no_opt_times[i] = duration;
    printf("第%d次运行时间: %lld 微秒 (%.3f秒)\n", i+1, duration, duration/1000000.0);
    usleep(100000);
}

printf("\n=== 优化版本测试 (Cache行对齐) ===\n");
for (int i = 0; i < iterations; i++) {
    long long start = get_current_time();
    cache_optimize_test(NULL);
    long long end = get_current_time();
    long long duration = end - start;
    opt_times[i] = duration;
    printf("第%d次运行时间: %lld 微秒 (%.3f秒)\n", i+1, duration, duration/1000000.0);
    usleep(100000);
}

long long avg_single = calculate_k_best(single_times, iterations, iterations);
long long avg_no_opt = calculate_k_best(no_opt_times, iterations, iterations);
long long avg_opt = calculate_k_best(opt_times, iterations, iterations);
long long k_best_single = calculate_k_best(single_times, iterations, iterations > 5 ? 5 : iterations);
long long k_best_no_opt = calculate_k_best(no_opt_times, iterations, iterations > 5 ? 5 : iterations);
long long k_best_opt = calculate_k_best(opt_times, iterations, iterations > 5 ? 5 : iterations);

printf("\n=====");

```

```

printf("                测试结果分析\n");
printf("=====\\n");

printf("平均运行时间:\\n");
printf("单线程基准:    %lld 微秒\\n", avg_single);
printf("未优化版本:    %lld 微秒\\n", avg_no_opt);
printf("优化版本:      %lld 微秒\\n", avg_opt);

printf("\\nK最佳运行时间:\\n");
printf("单线程基准:    %lld 微秒\\n", k_best_single);
printf("未优化版本:    %lld 微秒\\n", k_best_no_opt);
printf("优化版本:      %lld 微秒\\n", k_best_opt);

printf("\\n性能对比:\\n");
if (k_best_opt < k_best_no_opt) {
    double improvement = (1.0 - (double)k_best_opt / k_best_no_opt) * 100;
    printf("Cache优化带来性能提升: +%.2f%%\\n", improvement);
} else {
    double degradation = ((double)k_best_opt / k_best_no_opt - 1.0) * 100;
    printf("优化版本性能下降: -%.2f%%\\n", degradation);
}

printf("\\n相对于单线程:\\n");
double speedup_no_opt = (double)k_best_single / k_best_no_opt;
double speedup_opt = (double)k_best_single / k_best_opt;
printf("未优化版本并行加速比: %.2fx\\n", speedup_no_opt);
printf("优化版本并行加速比:    %.2fx\\n", speedup_opt);

free(single_times);
free(no_opt_times);
free(opt_times);
}

int main() {

    run_cache_test();

    return 0;
}

```

实验6-5 CPU 亲和力对并行程序影响 lab6-5.c

```

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>

#define APPLE_MAX_VALUE 50000000

struct apple {

```

```

    unsigned long long a;
    char padding1[128];
    unsigned long long b;
    char padding2[128];
};

long long get_current_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (long long)tv.tv_sec * 1000000 + tv.tv_usec;
}

void* calc_apple(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }
    return NULL;
}

void* calc_apple_b_no_lock(void* arg) {
    struct apple* test = (struct apple*)arg;
    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->b += sum;
    }
    return NULL;
}

void set_cpu_affinity(pthread_t thread, int cpu_id) {
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(cpu_id, &cpuset);

    int result = pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset);
    if (result != 0) {
        printf("设置CPU亲和性失败 (CPU %d)\n", cpu_id);
    }
}

void* no_affinity_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    test->a = 0;
    test->b = 0;

    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, calc_apple, test);
    pthread_create(&thread2, NULL, calc_apple_b_no_lock, test);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("无亲和性测试完成: a=%llu, b=%llu\n", test->a, test->b);
}

```

```

    free(test);
    return NULL;
}

void* with_affinity_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    test->a = 0;
    test->b = 0;

    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, calc_apple, test);
    pthread_create(&thread2, NULL, calc_apple_b_no_lock, test);

    set_cpu_affinity(thread1, 0);
    set_cpu_affinity(thread2, 1);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("有亲和性测试完成: a=%llu, b=%llu\n", test->a, test->b);
    free(test);
    return NULL;
}

void* single_thread_test(void* arg) {
    struct apple* test = (struct apple*)malloc(sizeof(struct apple));
    test->a = 0;
    test->b = 0;

    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->a += sum;
    }

    for (unsigned long long sum = 0; sum < APPLE_MAX_VALUE; sum++) {
        test->b += sum;
    }

    printf("单线程测试完成: a=%llu, b=%llu\n", test->a, test->b);
    free(test);
    return NULL;
}

int compare_long_long(const void* a, const void* b) {
    long long x = *(const long long*)a;
    long long y = *(const long long*)b;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

long long calculate_k_best(long long* times, int count, int k) {
    qsort(times, count, sizeof(long long), compare_long_long);
}

```

```

    long long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += times[i];
    }
    return sum / k;
}

void run_performance_test(const char* test_name, void* (*test_func)(void*), int iterations)
{
    printf("\n=== 开始测试: %s ===\n", test_name);

    long long* times = (long long*)malloc(iterations * sizeof(long long));

    for (int i = 0; i < iterations; i++) {
        long long start_time = get_current_time();
        test_func(NULL);
        long long end_time = get_current_time();
        long long duration = end_time - start_time;

        times[i] = duration;

        printf("  第%d次运行时间: %lld 微秒\n", i+1, duration);

        usleep(100000);
    }

    long long k_best_time = calculate_k_best(times, iterations, iterations > 5 ? 5 :
iterations);

    printf("=== %s 测试结果 ===\n", test_name);
    printf("最佳运行时间: %lld 微秒\n", times[0]);
    printf("平均运行时间: %lld 微秒\n", calculate_k_best(times, iterations, iterations));
    printf("K最佳运行时间: %lld 微秒\n", k_best_time);
    printf("=====\n");

    free(times);
}

void show_cpu_info() {
    printf("=== 系统CPU信息 ===\n");
    system("lscpu | grep -E '(CPU\\(s\\)|Core\\(s\\)|Socket\\(s\\)|Thread\\(s\\))'");
    printf("=== CPU架构信息 ===\n");
    system("lscpu | grep -E '(Architecture|Model name|MHz)'");
    printf("=====\n\n");
}

int main() {
    int iterations = 5;

    show_cpu_info();

    printf("开始CPU亲和性性能测试...\n");
    printf("每个测试将运行 %d 次, 取最佳%d次的平均值\n\n", iterations, iterations);
}

```



```
run_performance_test("单线程基准", single_thread_test, iterations);

run_performance_test("无CPU亲和性(两线程)", no_affinity_test, iterations);

run_performance_test("有CPU亲和性(两线程)", with_affinity_test, iterations);

return 0;
}
```

启动脚本 run.sh

```
#!/bin/bash

echo "编译所有实验程序..."
gcc lab1.c -o lab1
gcc lab2.c -o lab2
gcc lab3.c -o lab3
gcc lab4.c -o lab4
gcc lab5.c -o lab5
gcc lab6-1.c -o lab6-1
gcc lab6-2.c -o lab6-2
gcc lab6-3.c -o lab6-3
gcc lab6-4.c -o lab6-4
gcc lab6-5.c -o lab6-5

echo "运行实验..."
echo -e "\n实验1 - 进程创建与管理:"
./lab1

echo -e "\n实验2 - 线程创建与管理:"
./lab2

echo -e "\n实验3 - 进程间通信:"
./lab3

echo -e "\n实验4 - 线程间通信:"
./lab4

echo -e "\n实验5 - 同步与互斥:"
./lab5

echo -e "\n实验6.1 - 检测CPU信息:"
./lab6-1

echo -e "\n实验6.2 - 基础并行对比:"
./lab6-2

echo -e "\n实验6.3 - 加锁vs不加锁:"
./lab6-3

echo -e "\n实验6.4 - Cache优化:"
```

```
./lab6-4
```

```
echo -e "\n实验6.5 - CPU 亲和力对并行程序影响:"
```

```
./lab6-5
```